One ROM

One ROM CLI Manual

The command-line tool for managing One ROMs.

One ROM CLI v0.6.0

8 November 2026

Built from one-rom 1b189260 · source `docs/CLI-MANUAL.md`

Copyright © 2026 Piers Finlayson <piers@piers.rocks> · MIT licence

https://onerom.org

Contents

Introduction **5**
About This Document 5
New Breaking Changes **6**
One ROM Overview **7**
Background - ROM Replacements 7
Introduction to One ROM 7
Hardware 7
Important concepts 8
Types of One ROM Deployments 9
Minimal 9
Standard 9
Extended 9
Custom 9
CLI Guide **11**
Installation 11
Keeping the CLI up to date 11
How One ROM talks to the CLI 11
Identifying your device 12
Common workflows 12
Program a device from a config file 12
Program from `--slot` specifications 13
Program with a plugin 13
Plugin compatibility 13
Build firmware without flashing 14
Inspect a device 14
Read the live ROM image 14
Patch a running image 14
Identify a physical unit 15
Reset the host system after programming 15
Reserve a pin for another use 15
See what One ROM is doing with its GPIOs 16
Watch a device's log 16
Erase a device 16
Prepare a 16-bit ROM image 16
Device states 17
Global behaviour worth knowing 17
CLI Reference **18**
Synopsis 18
Global options 18
Command summary 18

© 2026 Piers Finlayson · onerom.org · MIT licence

scan ... 20
program ... 22
Source of the firmware (mutually exclusive groups) ... 22
Configuration metadata ... 22
Per-device overrides ... 23
Board, version and output ... 23
Reboot and flashing behaviour ... 24
save ... 25
inspect ... 27
inspect info ... 27
inspect telemetry ... 27
inspect slots ... 27
inspect peek ... 28
inspect gpio ... 29
inspect led ... 31
inspect rgb ... 32
inspect header ... 33
inspect socket ... 33
inspect otp ... 33
monitor ... 35
monitor log ... 35
console ... 37
control ... 38
control reboot ... 38
control led ... 38
control rgb ... 39
control standby ... 41
control poke ... 41
control reset ... 42
control select ... 43
control pin ... 43
control erase ... 44
update ... 46
update firmware ... 46
update slot ... 46
update plugin ... 47
update commit ... 47
clone ... 48
hardware ... 49
Board sizes ... 49
hardware commission ... 49
hardware request-signature ... 51
hardware sign ... 52
hardware set-size ... 53

hardware validate ... 54
image ... 56
image swap-bytes ... 56
image deinterleave ... 56
image convert ... 57
firmware ... 59
firmware build ... 59
firmware inspect ... 60
firmware releases ... 60
firmware download ... 61
firmware chips ... 61
plugin ... 62
chips ... 63
Flash usage ... 63
board ... 65
board list ... 65
board header ... 65
board socket ... 66
self ... 68
self check ... 68
self download ... 68
Top-level aliases ... 70
peek (top-level alias) ... 70
poke (top-level alias) ... 70
reboot (top-level alias) ... 70
ROM slot specification ... 71
Image transforms ... 73
16-bit ROM image byte ordering ... 74
Plugin specification ... 76
Pin values ... 77
Problems ... 78
Recovering a bricked One ROM ... 78
Situation ... 78
Booting into the bootloader ... 78
Checking the host can see it ... 79
Programming it again ... 79
Ice boards ... 79
Appendix: Breaking Change History ... 80
v0.6.0 ... 80
v0.5.0 ... 80
v0.4.0 ... 80
v0.3.0 ... 80

© 2026 Piers Finlayson · onerom.org · MIT licence

Introduction

`onerom` (`onerom.exe` on Windows) is the command-line tool for managing One ROM ROM emulators: discovering connected devices, building and flashing firmware, inspecting device state, and manipulating ROM image files.

About This Document

This One ROM CLI manual covers:

- **One ROM Overview** — what One ROM is, its hardware, and the vocabulary the rest of this manual uses.
- **CLI Guide** — installation and the common workflows.
- **CLI Reference** — every command, subcommand and option.
- **Problems** — symptoms and their fixes, including recovering a bricked One ROM.

> This manual documents the `onerom` CLI as of release v0.6.0. Board, chip and plugin lists shown in examples are illustrative — the set your build supports may differ. Run `onerom --version` to check your version, and `onerom board list` / `onerom chips` for the definitive lists your build knows about. Commands marked **(not yet supported)** are present in the CLI surface but not yet functional.

© 2026 Piers Finlayson · onerom.org · MIT licence

New Breaking Changes

These change what an existing command line does, so a script written against an earlier release is worth checking before upgrading.

The CLI is versioned `MAJOR.MINOR.PATCH`. While it is pre-v1.0.0, a change that breaks an existing command line lands in a minor release — v0.3.0 to v0.4.0 — and never in a patch. From v1.0.0 onwards such a change lands in a major release, and never in a minor or a patch.

- `program` and `firmware build` automatically add the USB plugin when a system plugin isn't specified. Use `--no-plugin` to build without any plugins.
- An unused signature from an earlier CLI fails with `hardware commission --signature`. One ROMs already commissioned aren't affected. Raise a new signing request, or sign again with `hardware sign`.
- `hardware commission` and `hardware sign` fail with a manufacturer name longer than 32 characters.
- `--config-name`, `--config-description`, `--save-config` and the per-device overrides are rejected when used with `program --firmware`. When used with `program --base-firmware` these, `--plugin` and `--no-plugin` are rejected unless `--config` or `--slot` is used. Previously they were ignored.
- `program --base-firmware` without `--config` or `--slot` requires `--no-config`.
- `console --no-echo` requires `--raw`.

Every release's breaking changes are collected in Appendix: Breaking Change History, at the end of this manual.

© 2026 Piers Finlayson · onerom.org · MIT licence

One ROM Overview

Background - ROM Replacements

ROM (Read-Only Memory) chips are used in a huge assortment of electronic devices to supply those systems with pre-programmed data - Operating System and BIOS images, character sets, programming languages, games, etc. On old systems these sometimes fail and need replacing, or users would like to upgrade their system, by replacing with a ROM with newer data. The original ROM chip is removed from the system and a replacement is installed in its place.

Traditional ROM replacements are built around EPROMs, EEPROMs and flash chips. Those are similar devices to the ROM being replaced - that is dumb devices that have a persistent memory store and some control lines - but with higher capacity than the original ROM and/or different chip select behaviour. The chip select logic is typically fixed with additional on-board logic.

Introduction to One ROM

One ROM takes a different approach - it consists of a microcontroller paired with flash and RAM, physically laid out on a PCB so that the original ROM's address, data and chip select lines connect to the MCU's general-purpose input/output (GPIO) pins. This makes One ROM a ROM emulator, also known as a Software Defined ROM.

Firmware is loaded to One ROM's flash, along with the ROM image(s) to be served. One ROM is installed in the system. On power on, it boots and first loads and runs its firmware, and then loads the ROM image(s) to be served into RAM, and serves them from there.

Because One ROM is microcontroller based, it is controlled by software rather than by hardwired, hardware-based logic. This makes One ROM the most powerful and flexible ROM replacement available. Nearly any original ROM of a particular form factor can be replaced by an equivalent One ROM.

One ROM's software is open source and has a modular architecture with a rich plugin API. This means One ROM is a platform that can be extended with new capabilities, not just a ROM replacement. Some examples of ways in which people have extended One ROM include:

- A comprehensive USB stack and interface to allow One ROM to be managed while running, built on the public plugin API.
- Communication between a retro system and a PC using One ROM's USB port.
- A retro system controlling its own reset line using one of One ROM's header pins.
- Reprogramming One ROM dynamically as part of a car's ECU tuning process.

Hardware

One ROM comes in multiple physical variants to support each supported ROM package size - 24, 28, 32 and 40 pin. All One ROMs replace original 600 mil (15.24mm) wide DIP packages. The supported ROM types are listed in COMPATIBILITY. Each pin variant has multiple hardware revisions, marked on the board.

One ROM Fire, based on the RP2350 microcontroller, is the current family and is available in all four sizes.

© 2026 Piers Finlayson · onerom.org · MIT licence

One ROM Ice is a deprecated 24 pin only family based on an STM32F4. It is supported on Base Firmware v0.6.x. New feature releases for One ROM Ice are not anticipated.

Important concepts

- **Base Firmware** - a binary including the core code that causes a One ROM to serve ROM images. Does not include any information about the One ROM hardware variant or the ROM images to serve. Should not be used on its own to program a One ROM device.
- **ROM image** - a binary blob that is the contents of a ROM to be served. Can be supplied to the programming tool as a raw binary file, or in other formats, including Intel HEX and Motorola S-record. The programming tool converts the ROM image to the format One ROM requires.
- **Slot** - a region of One ROM's flash holding a single ROM image, set of ROM images, or plugin.
- **Image selection** - the choice of which ROM image a One ROM serves. A One ROM can hold multiple images, selected at boot using the image select jumpers on the board.
- **Configuration** - the description of what a One ROM should do, provided to the programmer. It names the ROM images to serve, the chip type and chip select behaviour of each, and any settings.
- **Firmware** - the complete image a One ROM runs. Every One ROM is built from a combination of configuration and ROM images. A programmer turns them into firmware together with the Base Firmware, metadata and any plugins.
- **Programmer** - a tool that composes ROM images, configuration, Base Firmware and metadata into a complete image, and flashes it to the device. There are three - the web programmer, the CLI and Studio.
- **Metadata** - a binary block that describes the hardware properties of a physical One ROM device, and the properties of the ROM images to be served. The programmer generates this metadata from the configuration provided, and it is included in the firmware that is flashed to the device.
- **Bootloader** - a small program contained in One ROM's own read-only memory that allows One ROM to be reflashed with new firmware and metadata. The bootloader is always accessible, even if the firmware and metadata are corrupted or missing and, because it is ROM based itself, it cannot be corrupted or erased.
- **Header pins** - pins on the One ROM board exposing power, ground, the image select lines, the debug interface and spare pins for expansion. They allow One ROM to be wired to the system it is installed in, or to external hardware.
- **Plugin** - a binary blob consisting of code that extends One ROM's capabilities beyond those provided by the core firmware. The most common plugin is the System Plugin. A user chooses which plugins to include in their firmware (if any) at programming time. Up to two plugins can be included with a single One ROM firmware image - the System Plugin, and a User Plugin.
- **System Plugin** - a plugin that provides a USB stack and other capabilities to One ROM. One ROM's System Plugin is shipped alongside One ROM's Base Firmware. Its use is recommended for all users, except those who want to replace it with their own custom, replacement plugin.
- **User Plugin** - a plugin installed alongside the System Plugin providing additional capabilities to One ROM. User Plugins can be developed by anyone, and a number of them are shipped alongside Base Firmware and System Plugin. User Plugins require a System Plugin to be included. An example User

© 2026 Piers Finlayson · onerom.org · MIT licence

Plugin is One ROM's Host Control plugin, which allows One ROM to be controlled by the system it is installed in, with no extra wiring using the ROM Bus Control Protocol (RBCP).

- **Setting** - a named value that changes how a One ROM behaves rather than what it serves. Settings are configured by the user and written by the programmer. The status LED and the CPU frequency are example settings supported by the Base Firmware. Plugins may have their own settings.
- **ROM Bus Control Protocol (RBCP)** - a protocol that allows ROM emulators like One ROM to be controlled by a system it is installed in, with no extra wiring. Supported by One ROM's Host Control plugin.
- **Bricked One ROM** - a One ROM device that is non-functional because it has been flashed with corrupted firmware or metadata, metadata for a different physical One ROM device, no metadata, or firmware or metadata with some other issue. Bricked One ROMs can be recovered by entering One ROM's bootloader and reflashing them with valid firmware and metadata. It is very unlikely that a bricked One ROM cannot be recovered.

Types of One ROM Deployments

Minimal

Consists of:

- Base Firmware
- Metadata
- At least one ROM image

Contains no plugins. Serves the ROM image(s) to the system it is installed in, but cannot be managed while running.

In this deployment type, when One ROM is plugged in via USB it drops automatically into its bootloader, allowing the device to be reprogrammed, and simultaneously stops serving the ROM image(s) to the system it is installed in.

Standard

A Minimal Deployment plus One ROM's System Plugin, which includes a USB stack enabling comprehensive management of the device while it is running.

This is the recommended deployment type for most users.

Extended

A Standard Deployment plus a User Plugin. The User Plugin may be One ROM's own, or one written by a third party.

Custom

A Custom Deployment is one where one or more of the following holds:

- The physical One ROM device is not manufactured from a published design. It may be a derivative, or a fully custom design.
- The Base Firmware is a fork of or replacement for One ROM's Base Firmware.

© 2026 Piers Finlayson · onerom.org · MIT licence

- The System Plugin is replaced with a fork of or replacement for One ROM's System Plugin.

CLI Guide

Installation

Download the CLI from **https://onerom.org/cli**. Builds are provided for:

- Windows — x86 64-bit and ARM 64-bit
- macOS - a single universal build for Intel and Apple Silicon
- Ubuntu/Debian — x86 64-bit, and ARM 64-bit (also for Raspberry Pi)

The Windows and macOS builds are digitally signed. A sha256 checksum is published alongside every download so you can verify what you fetched.

Windows / macOS — unzip the archive and place the `onerom` (`onerom.exe`) executable in a folder on your `PATH`.

Linux — install the `.deb` as usual, which places `onerom` on your `PATH`:

```
sudo dpkg -i onerom-cli-x.y.z-1_amd64.deb
```

(replace `x.y.z` with the version, e.g. `0.1.10`, and `amd64` with `arm64` for the ARM/Pi build).

Windows SmartScreen — as a relatively new publisher, the first run may raise a *"Windows protected your PC"* dialog. Click **More info**, confirm the publisher reads *"Open Source Developer Piers Finlayson"*, then **Run anyway**.

Verify it runs:

```
onerom --version
```

Keeping the CLI up to date

The CLI does not check for updates on its own. `onerom self check` compares the current build against the newest release published for your platform, and `onerom self download` fetches it:

```
onerom self check
onerom self download
```

`self download` saves the same artifact you would get from https://onerom.org/cli — a `.deb` on Linux, a zip on Windows and macOS — verifies it against the SHA-256 published alongside it, and prints the install step for what it downloaded. You must install the new version.

How One ROM talks to the CLI

The CLI communicates with a One ROM over USB using picoboot (the Raspberry Pi bootloader protocol, extended by picobootx). A One ROM is reachable in two situations:

- **Running** — normal firmware is running and serving ROMs; its USB stack (provided by the system USB plugin) exposes the picobootx interface.

© 2026 Piers Finlayson · onerom.org · MIT licence

- **Stopped** — the device is in One ROM's bootloader (BOOTSEL). A bare RP2350 bootloader is also reachable here, which is how unprogrammed or bricked units are recovered.

Some commands work in either state; some require one specifically. Each reference entry notes when a device connection is required, and the state model is summarised under Device states.

Identifying your device

With **exactly one** One ROM or other One ROM device connected that the CLI recognises, you don't need to identify it — commands find it automatically and One ROM's board type is inferred from the device.

With **multiple** devices connected, select one with `--serial` (`-s`). It accepts `*` and `?` wildcards:

```
onerom --serial 'A1B2*' inspect info
```

`--serial` is **global**: it can appear at any level of the command line.

A device programmed with a serial override (`program --serial-override`) reports and is matched by that overridden serial while **Running**. When **Stopped** its USB stack comes from the RP2350 bootrom, so it falls back to reporting its chip ID — match it by chip ID (or reboot it to running) in that state.

Discover what's attached:

```
onerom scan
onerom scan --slots        # also list each device's ROM slots
```

Two situations need extra flags:

- **Unrecognised / unprogrammed / bricked** units: add `--unrecognised` (`-u`) and supply `--board`, since the board type can't be inferred. The unit must still answer on its picoboot USB interface — one that answers nothing is ignored either way, since it cannot be programmed. It reports no serial at all, so `--serial` cannot pick between two of them — attach one at a time, and see Recovering a bricked One ROM.
- **Non-standard USB IDs**: add `--vid-pid <VID:PID>` (hex), repeatable. When supplied, only the given VID/PID pairs are matched.

`--board` (`-b`) can also be given on most commands to **override** the detected board type.

Common workflows

Program a device from a config file

The primary workflow. Build firmware from a JSON config and flash it in one step:

```
onerom program --config c64.json
```

`program` builds *and* flashes. To build a firmware binary **without** flashing, use `firmware build` instead. To build and also keep the binary while flashing, add `--output`:

```
onerom program --config c64.json --out firmware.bin
```

© 2026 Piers Finlayson · onerom.org · MIT licence

Program from `--slot` specifications

Instead of a config file, describe each ROM slot inline. Repeat `--slot` per slot. The required chip-select lines depend on the chip type (e.g. a 2332 needs `cs1` and `cs2`):

```
onerom program --board fire-24-e \
    --slot file=kernal.bin,type=2364,cs1=active-low \
    --slot file=basic.bin,type=2364,cs1=active-low
```

The full slot spec grammar is documented under ROM slot specification — it covers CS polarity, size handling, per-slot CPU frequency/voltage, the status LED and 16-bit forcing.

Program with a plugin

Plugins masquerade as ROMs. At most one system plugin and one user plugin are supported. A user plugin requires a system plugin. A system plugin uses slot 0 and a user plugin in slot 1.

The USB plugin is automatically added when a system plugin isn't specified. Therefore this command adds both the USB plugin and the host-control plugin:

```
onerom program --board fire-24-e \
    --slot file=kernal.bin,type=2364,cs1=active-low \
    --plugin host-control
```

`--plugin` may also be combined with `--config`. The plugins are inserted ahead of the config's ROM slots (which shift up accordingly), so you can add a plugin to a stock config without editing it:

```
onerom program --config c64.json --plugin host-control
```

It is an error if the config already defines a plugin of its own — remove it from the config, or drop `--plugin`.

Use `--no-plugin` to build without plugins. Where the USB plugin doesn't have a version that supports the firmware, the image is built without it:

```
$ onerom firmware build --board fire-24-c --version 0.5.10 --slot
file=kernal.bin,type=2364,cs1=active-low --out firmware.bin
No version of plugin 'usb' supports the selected firmware version 0.5.10.
  Building without a plugin.
Firmware written to firmware.bin
```

Plugin spec forms are listed under Plugin specification.

Plugin compatibility

Every plugin going into an image is checked against the compatibility window published on the images server, whether it arrived via `--plugin` or the a config file.

An incompatible plugin version is refused:

© 2026 Piers Finlayson · onerom.org · MIT licence

```
$ onerom firmware build --config usb-0.1.2.json --board fire-24-a --output fw.bin
Failed to execute command.
Plugin 'usb' version '0.1.2' is not compatible with firmware 0.7.0 or later.
  The selected firmware version is 0.7.1.
  Plugin version 0.2.1 supports it: https://images.onerom.org/plugins/system/usb/v0.2.1/
plugin.bin
```

`--verbose` reports a plugin that passed:

```
Plugin 'usb' v0.2.1 is compatible with firmware
```

A plugin loaded from a local path, or from any URL that is not an official images-server one, has no published compatibility to check against and is built in as-is. Under `--verbose`:

```
Plugin /home/me/my-plugin.bin is not an official One ROM plugin - no published compatibility to
check
```

If the images server cannot be reached the build is not blocked — an offline build of an otherwise valid config still works — but a `Warning:` line naming the plugin says the check was skipped. The USB plugin that is added automatically is downloaded from the images server, so an offline build needs `--no-plugin`.

Build firmware without flashing

```
onerom firmware build --config c64.json --board fire-24-e --out firmware.bin
```

Inspect a device

```
onerom inspect info      # serial, name, board, MCU, firmware version, hw revision
onerom inspect slots     # ROM slots, with the selected one marked
```

Read the live ROM image

Read what the device would serve for a given logical ROM address (device must be running). The top-level `peek` is an alias for `inspect peek live`:

```
onerom peek --address 0x100 --length 64
onerom peek --address 0 --length 8192 --output rom-image.bin
```

Patch a running image

`poke live` writes to the ROM image currently being served, at a logical ROM offset. Changes are transient — lost on reboot. The top-level `poke` is an alias for `control poke live`:

```
onerom poke --address 0x100 --byte 0xEA
onerom poke --address 0 --input patch.bin
```

For file patches you can write only the differing bytes, and preview first:

© 2026 Piers Finlayson · onerom.org · MIT licence

```
onerom poke --input patch.bin --delta --dry-run
onerom poke --input patch.bin --delta
```

Identify a physical unit

Make the status LED beacon so you can spot which board is which:

```
onerom control led beacon
```

Reset the host system after programming

If a header pin is wired to the reset line of the machine One ROM is installed in, One ROM can pulse that pin low after programming. The host then boots the image just flashed:

```
onerom program --config c64.json --reserve-pin sel_c --reset-host sel_c
```

`--reset-host` waits for the One ROM to return to the USB bus, then sends the pulse. `--reserve-pin` stops One ROM using the pin — see Reserve a pin for another use. To reset a host without programming it, or to choose the length of the pulse, use `control reset`:

```
onerom control reset --pin sel_c
onerom control reset --pin sel_c --hold 500
```

The pin is typically `X1`, `X2` or an image select pin. The device times the pulse so an interrupted CLI cannot leave the host held in reset. See `control reset`, and `control pin` for driving a pin to any state.

Reserve a pin for another use

Reserve a pin for another use, for example a pin connected to the host's reset line, when the image is built:

```
onerom program --config c64.json --reserve-pin sel_c
```

Repeat `--reserve-pin` for each pin. Only image select pins and X pins can be reserved. Reserving a pin requires firmware v0.8.0 or later.

One ROM does not pull or read a reserved image select pin at boot. It reads the image number from the remaining image select pins in order, so with `SEL_C` reserved `SEL_D` selects image 4. `--verbose` lists the jumpers that select each image, with a mark for each column of the board's jumper header:

```
Reserved pins: SEL_C

Images:
      D C B A
0: [· · · ·]  2364  no image select jumpers
1: [· · · ■]  2364  SEL_A jumpered
2: [· · ■ ·]  2364  SEL_B jumpered
3: [· · ■ ■]  2364  SEL_A and SEL_B jumpered
4: [· ■ · ·]  2364  SEL_D jumpered
```

A slot that uses a reserved pin fails to build, for example where an X pin is reserved, but it is required to a banked or multi slot config.

© 2026 Piers Finlayson · onerom.org · MIT licence

It is possible to build configs where a slot cannot be selected by the remaining image select jumpers. That is reported:

```
Warning: slot 4 cannot be selected by jumpers. SEL_A and SEL_B provide 4 combinations for 5
slots.
```

`inspect slots`, `scan --slots` and `firmware inspect` list the reserved pins, and `inspect gpio` marks them `reserved`.

See what One ROM is doing with its GPIOs

```
onerom inspect gpio
```

One row per MCU GPIO: everything that GPIO is — its ROM socket signal under the image being served, the board peripheral it drives, the header pin it surfaces on — plus direction, level, 5V tolerance, and what One ROM itself is using it for. GPIOs connected to nothing are omitted unless you pass `--all`. Useful before driving a pin — see `inspect gpio`.

Watch a device's log

```
onerom monitor log
```

Prints the One ROM's firmware and plugin logging as it is written, and starts with whatever it has logged since anything last listened — so the boot log is still there when you attach. Needs a running One ROM with the USB system plugin. Add `--output <FILE>` to keep a transcript, or use `onerom program --config c64.json --follow` to go straight from programming to watching. See `monitor log`.

Erase a device

Erase flash. This is best done while stopped; by default the command reboots the device into the required state first. A fully erased unit falls back to One ROM's bootloader and is then reprogrammed with `--unrecognised` + `--board`, as Recovering a bricked One ROM describes:

```
onerom control erase --all
onerom control erase --offset 0x20000 --length 0x1000
```

Read `control erase` before using it while the device is running — erasing the core firmware or system plugin will take down the USB stack.

Prepare a 16-bit ROM image

16-bit ROM types (e.g. 27C400) may need their byte pairs swapped to match the order One ROM expects. Either rewrite the file first:

```
onerom image swap-bytes --input kick.bin --output kick-swapped.bin
```

or let the build do it, leaving the source file untouched:

```
onerom program --slot file=kick.bin,type=27C400,transform=swap_bytes
```

© 2026 Piers Finlayson · onerom.org · MIT licence

If your image interleaves several devices in one file — a 32-bit ROM set, say — `deinterleave` extracts a single lane. See Image transforms for the full set and how they compose.

Device states

Many commands reboot the device and, by default, pause briefly afterwards to let it re-enumerate on the USB bus.

- **Running** (default reboot target) — firmware active. One ROM is serving ROMs.
- **Stopped** — One ROM/RP2350 bootloader (BOOTSEL), required for some flash operations.

Common controls, where a command supports them:

- `--running` (`-r`) / `--stopped` (`-p`) — choose the post-operation state.
- `--no-reboot` — leave the device as-is.
- `--fast` — skip the re-enumeration pause.
- `--msd` (`-m`) — mount the mass-storage device when rebooting into stopped mode.

Global behaviour worth knowing

- `--yes` (`-y`) auto-confirms all prompts (non-interactive use). It also suppresses the confirmation otherwise required for CPU frequencies above 150 MHz and voltages above 1.10 V in slot specs. Use with care.
- `--verbose` (`-v`) prints device-selection progress and other detail.
- `--log-level <LEVEL>` sets log verbosity; defaults to `warn`. Run `onerom --help` for the accepted levels.

© 2026 Piers Finlayson · onerom.org · MIT licence

CLI Reference

Synopsis

```
onerom [GLOBAL OPTIONS] <COMMAND> [ARGS]
```

Global options

Available on every command (they are `global` in clap terms and may appear at any level).

Option	Description
`--serial, -s <DEVICE>`	Select a One ROM by serial number. Required when multiple are connected; auto-selected when exactly one is present. Accepts `*` and `?` wildcards.
`--vid-pid <VID:PID>` (alias `--id`)	USB vendor/product ID pair in hex (e.g. `1234:abcd`). Repeatable; when given, only these pairs are matched. Use with `--unrecognised`.
`--unrecognised, -u` (alias `--unrecognized`)	Allow management of unrecognised/unprogrammed/bricked RP2350 boards. The unit must still answer on its picoboot USB interface — a device that answers nothing is ignored either way. Use with caution — permits programming any attached RP2350 board. See Recovering a bricked One ROM.
`--yes, -y`	Auto-confirm all prompts. Also suppresses the over-limit CPU frequency/voltage confirmations.
`--verbose, -v`	Enable verbose output.
`--log-level <LEVEL>`	Set log level. Defaults to `warn`.
`--version, -V`	Print version.
`--help, -h`	Print help. Works on any subcommand.

Most commands accept `--board` (`-b`) to identify or override the board type, and rely on `--serial` (global) to pick a specific device.

Ice (STM32) boards are recognised, but the CLI cannot scan, program or build firmware for them. Where `--board` reaches a device or builds an image, naming an Ice board is an error rather than a later failure. Commands that only report hardware take them; each command's own entry below states what it accepts, and `board list` shows which boards are which.

Command summary

Command	Purpose	Device required
`scan`	Discover connected One ROMs	No
`program`	Build and flash firmware to a One ROM	Yes
`save`	Save a One ROM's configuration and ROM images to files	Varies
`inspect`	Read-only device state and information	Yes

© 2026 Piers Finlayson · onerom.org · MIT licence

Command	Purpose	Device required
`monitor`	Watch a running One ROM as it works	Yes
`console`	Send data to and receive data from the machine One ROM is fitted in	Yes
`control`	Transient (non-persistent) device actions	Yes
`update`	Persistent device modifications	Yes
`clone`	Copy a One ROM or firmware file to another One ROM	Yes
`hardware`	Commission and check a One ROM's hardware	Varies
`image`	ROM image file manipulation	No
`firmware`	Build, inspect and manage firmware binaries	Varies
`plugin`	List available plugins	No
`chips`	List supported chip types and their flash usage	No
`board`	List board types, or draw a board's pin header / socket	No
`self`	Check for and download new releases of the CLI itself	No
`peek`	Alias for `inspect peek live`	Yes
`poke`	Alias for `control poke live`	Yes
`reboot`	Alias for `control reboot`	Yes

© 2026 Piers Finlayson · onerom.org · MIT licence

scan

Discover and list connected One ROMs and One ROM Labs. Each device's line contains its:

- board type, followed by its size unless the size is M
- firmware version
- state
- serial

`--verbose` (`-v`) adds each device's:

- MCU variant
- chip ID
- commissioning information

```
onerom scan
onerom scan --board fire-24-e
onerom scan --slots
```

Option	Description
`--board <BOARD>`	Only show devices matching this board type. Conflicts with `--list-boards`. Must be a Fire board — a scan cannot find an Ice board.
`--list-boards`	List the known board types, the same listing as `board list`.
`--slots` (alias `--slot`)	Also show the ROM slot contents for each device found. Conflicts with `--list-boards`.

Example output:

```
Scanning ...
found 1 connected device:
  One ROM Fire 28 C - Firmware: v0.7.2 State: Running Serial: FC9D67248E8E8023
```

With `--verbose` on a `fire-40-b` commissioned by Acme Retro:

```
$ onerom scan --verbose
Scanning ...
found 1 connected device:
  One ROM Fire 40 B - Firmware: v0.8.0 State: Stopped Serial: DE3F9C232F655B6B
    MCU: RP235xB Chip ID: DE3F9C232F655B6B
    Commissioned: yes
      Board type:   fire-40-b
      Manufacturer: Acme Retro
      Date:         2026-09-29
      Signing key:  Acme Retro (256)
```

One ROM Lab shows its board type, or `(board not set)`. `--slots` doesn't show any additional output.

```
Scanning ...
found 1 connected device:
  One ROM Lab (board not set) - Firmware: v0.4.0 State: Running Serial: 62CD9AE3C0771A7E
```

© 2026 Piers Finlayson · onerom.org · MIT licence

Device required: no.

program

Build a firmware image (from a config file, inline `--slot` specs, or a supplied binary) and flash it to a connected One ROM. This is the primary workflow. `onerom firmware program` is an alias for this command.

```
onerom program --config c64.json
onerom program --config https://images.onerom.org/configs/c64.json
onerom program --serial '5*' --config c64.json
onerom program --board fire-24-e \
    --slot file=kernal.bin,type=2364,cs1=active-low \
    --slot file=basic.bin,type=2364,cs1=active-low
onerom program --firmware firmware.bin
onerom program --config c64.json --out firmware.bin
```

Source of the firmware (mutually exclusive groups)

Option	Description
`--config, -j <FILE>` (aliases `--config-file`, `--config-json`, `--json`)	ROM configuration JSON file or http(s) URL. Conflicts with `--slot`, `--config-name`, `--config-description`, `--save-config`, `--no-config`, `--firmware`.
`--slot <SPEC>` (alias `--rom`)	ROM slot specification; repeatable. See ROM slot specification. Conflicts with `--config`, `--no-config`, `--firmware`.
`--firmware <FILE>` (alias `--fw`)	Flash a pre-built complete firmware binary directly. Conflicts with `--config`, `--slot`, `--base-firmware` and `--plugin` because a pre-built firmware already contains all ROMs/plugins. Also conflicts with `--version`, `--config-name`, `--config-description`, `--save-config` and the per-device overrides.
`--base-firmware <FILE>`	Use a local minimal firmware instead of downloading. Without `--config` or `--slot` it requires `--no-config` and is flashed as is. It then can't be used with `--plugin`, `--no-plugin`, `--config-name`, `--config-description`, `--save-config` or the per-device overrides. Conflicts with `--firmware`, `--version`.
`--no-config`	Confirm flashing a base firmware with no ROM configuration. Only valid with `--config-name` and/or `--config-description`. Conflicts with `--config`, `--slot`, `--firmware`, and the config-override options below.

Configuration metadata

Option	Description
`--plugin <SPEC>`	Plugin specification. Repeatable. See Plugin specification. Can be used with `--config` if the config doesn't contain a plugin. Conflicts with `--firmware`.
`--no-plugin`	Build without any plugins. Conflicts with `--plugin` and `--firmware`.
`--config-name <NAME>`	Name for the generated ROM configuration. Conflicts with `--config`.
`--config-description <DESC>` (aliases `--desc`, `--description`)	Description for the generated configuration. Defaults to *"Created by the One ROM CLI"*. Conflicts with `--config`.

© 2026 Piers Finlayson · onerom.org · MIT licence

Option	Description
`--save-config <FILE>`	Save the generated configuration to JSON. Only valid with `--slot` or `--no-config`. Conflicts with `--config`. `--plugin usb` is saved as `"plugin": "usb"`.

Per-device overrides

These are rejected when used with `--no-config` or `--firmware`. When used with `--base-firmware` they're rejected unless `--config` or `--slot` is used.

Option	Description
`--instance-name <NAME>` (aliases `--name`, `--onerom`, `--one-rom`, `--onerom-name`, `--one-rom-name`, `--instance_name`)	Assign this One ROM a name.
`--serial-override <SERIAL>`	Override the device's reported serial number.
`--logging [BOOL]` (aliases `--boot-logging`)	Enable boot logging. Takes an optional boolean; bare flag means `true`.
`--disable-swd [BOOL]` (aliases `--swd-disable`)	Shut SWD down before ROM serving starts, so debug port accesses to SRAM don't steal cycles from the serving DMAs. SWD is available for the whole of boot — including boot logging — and goes off until the next reset. Nothing is logged past that point, and plugins get no logging. This is not a debug lockout: the boot ROM runs before the One ROM firmware does, and BOOTSEL/PICOBOOT are unaffected. Optional boolean; bare flag means `true`.
`--turbo-boot [BOOL]`	Enable turbo boot — starts serving faster by not reading the image select jumpers, so the first non-plugin slot is always the one served. More than one non-plugin slot is refused unless `--force` is provided. Optional boolean; bare flag means `true`.
`--reserve-pin <PIN>` (aliases `--reserved-pin`, `--reserved_pins`)	Reserve a pin for another use, for example a pin connected to a host's reset line. Repeat for each reserved pin. Requires firmware v0.8.0 or later. See Reserve a pin for another use.
`--addr-mon-min-cs-act-ns <NS>` (alias `--addr_mon_min_cs_act_ns`)	Set the minimum time in nanoseconds chip select has to stay active for the address monitor to capture an access. A pulse slightly longer than this may also not be captured. Unset, a pulse of 40ns or less isn't captured, or 20ns or less for a 23QL384, at the default CPU frequency of 150MHz. With a lower value, shorter pulses can be captured. The value may need tuning for a particular host. A per-slot value overrides it. The range is 0 to 2540. Requires firmware v0.8.1 or later.

Board, version and output

Option	Description
`--board, -b <BOARD>`	Target board type. Inferred from the connected device if omitted.
`--version <VERSION>`	Firmware version to build against. Defaults to the latest release. Conflicts with `--firmware`, `--base-firmware`.
`--output, -o <FILE>` (alias `--out`)	Also write the built firmware to this file while flashing.

© 2026 Piers Finlayson · onerom.org · MIT licence

Reboot and flashing behaviour

Option	Description
`--stopped, -p`	After flashing, reboot into stopped (bootloader) mode. Conflicts with `--running`.
`--running, -r`	After flashing, reboot into running mode (the default). Conflicts with `--stopped`.
`--no-reboot`	Do not reboot after flashing. Conflicts with `--stopped`.
`--fast`	Skip the re-enumeration pause after the final reboot. Conflicts with `--no-reboot`.
`--msd, -m`	Mount mass storage when rebooting into stopped mode.
`--verify`	Verify flash by reading back after programming.
`--force, -f`	Continue despite non-fatal problems, reporting each as a warning.
`--batch` (aliases `--multiple`, `--multi`)	Program multiple devices, pausing for confirmation between each. Every board is programmed with the same configuration as the first.
`--scan-slots`	After programming, run `onerom scan --slots` to show the result. Conflicts with `--fast`.
`--follow`	After programming, monitor the One ROM's log, as `monitor log` does. Runs after `--scan-slots`, and only once the One ROM is back on the USB bus, so it shows the boot log of the firmware just flashed. Refused before anything is flashed if the image has no USB system plugin, since such a One ROM leaves the bus as soon as it serves. Conflicts with `--fast`, `--stopped`, `--no-reboot`, `--no-plugin` and `--batch`.
`--reset-host <PIN>` (alias `--host-reset`)	After programming, pulse this pin low to reset the host system, as `control reset` does. Written as `gpio<N>` or as a header pin (see Pin values). Runs after `--scan-slots` and before `--follow`, once the One ROM is back on the USB bus, and for each device in a `--batch`. The pulse is 100ms. Use `control reset` for a different hold time. A warning is displayed if the pin isn't reserved. Conflicts with `--fast`, `--stopped`, `--no-reboot` and `--no-plugin`.

An image is built for the One ROM's size. For a commissioned One ROM it must also be built for the board type it is commissioned as, unless `--force` is used.

Device required: yes.

© 2026 Piers Finlayson · onerom.org · MIT licence

save

Save a One ROM's configuration and ROM images to files.

Use `--path` to save the config and every ROM image, or `--slot` to save one ROM image. The file paths in `config.json` start with the `--path` directory. Use `onerom program --config` in the directory where `onerom save` was used. A published plugin is listed in `config.json` by name and isn't saved.

```
onerom save --slot 4 --output basic.bin
onerom save --slot 3 --chip 1 --output char.bin
onerom save --path c64
onerom save --firmware old.bin --path c64
```

With `--path`, each ROM image is saved as `slot<N>[-chip<M>][-<name>].bin`, for example `slot2-kernal.bin`:

- `-chip<M>` is added for a slot with multiple chips.
- `<name>` is the ROM image's original file name without its extension.

A plugin that isn't published is saved as `<type>-plugin[-<name>].bin`, for example `system-plugin.bin`.

A saved ROM image may differ from the original file as it includes any padding or duplication of the original image.

An example showing a fire-24-f firmware file with:

- two single 2364 ROM images `a.bin` and `b.bin`
- a banked pair `c.bin` and `d.bin`:

```
$ onerom save --firmware image.bin --path c64
Saved c64/slot0-a.bin
Saved c64/slot1-b.bin
Saved c64/slot2-chip0-c.bin
Saved c64/slot2-chip1-d.bin
Saved c64/config.json
```

The command fails if a file it would write already exists. `--force` overwrites.

```
$ onerom save --firmware image.bin --path c64
Failed to execute command.
File already exists: c64/config.json
  Use --force to overwrite it.
```

A number of fields that can be put in a config file don't exist in the firmware, so are not written. These include:

- name
- descriptions
- notes
- categories
- licences

© 2026 Piers Finlayson · onerom.org · MIT licence

Option	Description
`--slot <INDEX>`	Slot to save the ROM image from. Slots are numbered as in `onerom inspect slots`. Requires `--output`.
`--chip <INDEX>`	Chip to save from a slot with multiple chips. Chips are numbered from 0. Requires `--slot`.
`--output, -o <FILE>` (alias `--out`)	File to save the ROM image to. Requires `--slot`.
`--path <DIR>`	Directory to save `config.json` and a file for each ROM image to. Created if it doesn't exist.
`--firmware <FILE>` (aliases `--fw`, `--in`, `--input`)	Firmware file to read instead of a connected One ROM.
`--force, -f`	Overwrite existing files.

Device required: yes, unless `--firmware` is provided.

© 2026 Piers Finlayson · onerom.org · MIT licence

inspect

Read-only inspection of a connected One ROM.

```
onerom inspect <COMMAND>
```

Subcommand	Purpose	Device required
`info`	Identity and configuration	Yes
`telemetry`	Runtime telemetry **(not yet supported)**	Yes
`slots`	List ROM slots	Yes
`peek`	Read SRAM or the live ROM image	Yes
`gpio`	Show what each GPIO is and what it is doing	Yes (running)
`header`	Draw the device board's pin header	Yes
`socket`	Draw the device board's ROM socket pinout	Yes
`otp`	Print the contents of One ROM's OTP	Yes

inspect info

Print the device's identity and configuration information.

The command then dumps the device's parsed structures as JSON. A `Parser notes:` block follows it when a structure is missing, or when the CLI is parsing a firmware newer than it understands. One ROM Lab's own structures sit under the dump's `metadata` and `runtime` keys.

```
onerom inspect info
onerom --serial 1234abcd inspect info
```

inspect telemetry

Access counts, timing statistics and other runtime metrics. **(not yet supported)**

Option	Description
`--json`	Output telemetry as JSON.

inspect slots

List the plugins and ROM slots stored on the device. No options. On a running One ROM the slot selected at boot is marked `active`, or `standby` while One ROM is in standby:

```
  Configured with 3 slots - Slot 0 is selected (standby)
  Slot 0 (selected, standby):
```

Any reserved pins are displayed:

```
  Configured with 3 slots - Slot 0 is selected (active)
  Reserved pins: SEL_C, X1
```

`scan --slots` displays the same information.

inspect peek

Read device memory. `peek memory` reads SRAM (and, in stopped state, page-aligned flash); `peek live` reads the ROM image currently being served.

```
onerom inspect peek <COMMAND>
```

inspect peek live

Read from the live ROM image at a **logical** ROM offset (starting at 0). The device must be running. Also reachable as the top-level `peek`.

```
onerom inspect peek live --address 0x100 --length 64
onerom inspect peek live --address 0 --length 8192 --output rom-image.bin
```

Option	Description
`--address, -a <ADDRESS>` (alias `--addr`)	Logical ROM address to read from, starting at 0. Decimal or `0x` hex. Default `0`. On a One ROM serving half a 27C080, address 0 is the start of that half.
`--length, -l <LENGTH>` (aliases `--len`, `--size`)	Number of bytes to read. Decimal or hex. If omitted, reads to the end of the live image.
`--output, -o <FILE>` (alias `--out`)	Save the data to this file.

A line is output preceding the data, saying what was read.

```
$ onerom inspect peek live --address 0x100 --length 16
Read 16 byte(s) from live ROM offset 0x00000100
0x0100  00 01 02 03  04 05 06 07  08 09 0a 0b  0c 0d 0e 0f  |................|
```

In standby the line ends `(standby)`.

inspect peek memory

Read One ROM's SRAM. Most addresses reachable via PICOBOOT can be queried. In stopped state, SRAM holds no meaningful data, and flash reads must be aligned to flash page boundaries.

```
onerom inspect peek memory --address 0x20000000 --length 128
onerom inspect peek memory --address 0x10000000 --length 8192 --output flash-start.bin
```

Option	Description
`--address, -a <ADDRESS>` (alias `--addr`)	Address to read from. Decimal or `0x` hex.
`--length, -l <LENGTH>` (aliases `--len`, `--size`)	Number of bytes to read. Decimal or hex.
`--output, -o <FILE>` (alias `--out`)	Save the data to this file.

© 2026 Piers Finlayson · onerom.org · MIT licence

inspect gpio

Show, one row per MCU GPIO, what that GPIO is on this board and what One ROM is currently doing with it.

The device must be **running** with the USB system plugin: One ROM's own command handler lives in that plugin, and a stopped device is in the RP2350 bootloader where it does not exist. See Device states.

```
onerom inspect gpio
onerom inspect gpio --all
onerom inspect gpio --pin gpio9
onerom inspect gpio --pin x1
```

Option	Description
`--pin <PIN>`	Show only this pin, a header pin or an MCU GPIO written `gpio<N>` (see Pin values). Conflicts with `--all`.
`--board <BOARD>`	Board type, overriding what the device reports. Only needed to resolve a `--pin` header pin on a board this build does not recognise.
`--all`	Also list GPIOs with no function at all. By default only GPIOs connected to something are shown.

By default the table lists only the GPIOs connected to **something** — a ROM socket signal, a board peripheral or a header pin. On a 48-GPIO board some of the GPIOs are connected to nothing, and listing them may bury the rows worth reading. A line beneath the table says how many were omitted. `--all` lists every GPIO. Note the filter is on what the GPIO *is*, not on what the device reports using it for: the `X1` / `X2` and image-select pins report `free` and are exactly what you read this table to find, so they always appear.

The number of GPIOs the device has is its own — 30 on an RP2350A, 48 on an RP2350B.

Columns:

Column	Meaning
`GPIO`	MCU GPIO number.
`Function`	Everything this GPIO is, comma-separated in a fixed order: its ROM socket signal under the image being served (`A5`, `D3`, `CS1`, `BYTE/VPP`), then the board peripheral (`Status LED`, `RGB LED`, `USB VBUS`, `ext flash CS`), then the header pin (`X1`, `X2`, `SEL_A`). – if the GPIO is connected to nothing.
`Dir`	`out` if the pin's output driver is enabled, `in` if not.
`Level`	The GPIO's level, `0` or `1`: what an `out` pin is driving, what an `in` pin reads.
`Max V`	`5V` if the GPIO is 5V-tolerant, `3V3` if it is an RP2350 ADC pin and therefore 3.3V-only, `?` if the board is not characterised.
`Current use`	What One ROM is using the GPIO for: `free`, `serving (read)`, `serving (driven)`, `standby`, `input forced`, `system`, `plugin` or `reserved`.

`Function` lists everything that applies rather than stopping at the first match, so a GPIO that is genuinely two things says so: on a `fire-24-f` the Status LED and the RGB LED are the same GPIO, and it reads `Status LED, RGB LED`. Names that would repeat are shown once — on a 32-pin board a high address line is both the socket's `A17` and the `A17` header pin, which is one net.

© 2026 Piers Finlayson · onerom.org · MIT licence

`Function` names only what a **GPIO** is. A header pin may carry more than the GPIO wired to it — on a Fire 24/28 board the `SEL_C` and `SEL_D` pins sit on the SWCLK and SWDIO nets — but SWCLK and SWDIO are dedicated RP2350 pins with no GPIO of their own, so they do not appear here. Run `inspect header` for the pin-by-pin view, which shows every role each pin carries.

`Current use` displays what a GPIO is being used for:

- `free`: not used.
- `reserved`: reserved for another use, and not used.
- `input forced`: not read, so driving it has no effect on serving.
- `serving (read)`: an address, chip select or `/BYTE` pin. Driving it changes what One ROM serves until the pin is released.
- `serving (driven)`: a data pin. Driving it stops serving until One ROM reboots.
- `standby`: a data pin while One ROM is in standby. Driving it breaks serving until One ROM reboots.
- `system`: a board peripheral, for example the status LED.
- `plugin`: used by a plugin.

A `serving`, `standby`, `system` or `plugin` pin can only be driven with `--force`. See `control pin`.

In standby the title ends `(standby)`, for example `serving 2364 (standby)`.

With `--verbose` (`-v`) the table is followed by a legend restating where each column comes from, what `Dir` and `Level` mean and what the `3V3` / `5V` tags mean. Nothing is lost without it: the cost of taking a serving pin over is stated at the point of action by `control pin` itself.

A board revision or ROM type this build does not recognise costs the derived names, not the listing: `Function` falls back to `-` (or, for a socket pin whose chip type is unknown, `socket pin <N>`), and with no recognised board at all nothing is filtered out, since nothing can be ruled out. On a board with no pin-header descriptor, header pins come from the board's pin assignments alone and `--verbose` says so beneath the table.

On a Fire 28 (rev C) serving a 27512:

© 2026 Piers Finlayson · onerom.org · MIT licence

```
One ROM Fire 28 C – Firmware: v0.7.2 State: Running Serial: 2E4A671D1C92AE5C

GPIO state  ·  One ROM Fire 28 (rev C)  ·  RP235xB  ·  serving 27512

  GPIO  Function    Dir  Level  Max V  Current use
  ────  ──────────  ───  ─────  ─────  ────────────────
  0     D0          out  0      5V     serving (driven)
  1     D1          out  0      5V     serving (driven)
  2     D2          out  1      5V     serving (driven)
  3     D3          out  1      5V     serving (driven)
  4     D4          out  1      5V     serving (driven)
  5     D5          out  1      5V     serving (driven)
  6     D6          out  1      5V     serving (driven)
  7     D7          out  0      5V     serving (driven)
  8     X2          in   0      5V     free
  9     X1          in   0      5V     free
  10    CE/PE       in   0      5V     serving (read)
  11    OE/VPP      in   0      5V     serving (read)
  12    A14         in   0      5V     serving (read)
  13    A10         in   0      5V     serving (read)
  14    A11         in   0      5V     serving (read)
  15    A9          in   0      5V     serving (read)
  16    A8          in   0      5V     serving (read)
  17    A13         in   0      5V     serving (read)
  18    A15         in   0      5V     serving (read)
  19    A12         in   0      5V     serving (read)
  20    A7          in   0      5V     serving (read)
  21    A6          in   0      5V     serving (read)
  22    A5          in   0      5V     serving (read)
  23    A4          in   0      5V     serving (read)
  24    A3          in   0      5V     serving (read)
  25    A2          in   0      5V     serving (read)
  26    A1          in   0      5V     serving (read)
  27    A0          in   0      5V     serving (read)
  38    SEL_C       in   1      5V     free
  39    SEL_D       in   1      5V     free
  40    SEL_A       in   0      3V3    free
  41    SEL_B       in   0      3V3    free
  44    RGB LED     out  0      3V3    system
  45    Status LED  out  0      3V3    system
  46    USB VBUS    in   1      3V3    system

  13 GPIOs with no function are hidden – use --all to show them.
```

inspect led

Show what the status LED is doing now — the mode it is in, how fast it is running, and which GPIO it is on. Use `inspect rgb` for the RGB LED some models support.

```
onerom inspect led
```

On a `fire-28-c` running `onerom control led flame --period 900`:

```
Status LED:
  Mode:       flame
  Period:     900ms
```

`Period` appears only for the modes that repeat.

`--verbose` adds the GPIO the LED is on, and says so where the board wires both LEDs to one pin — a `fire-24-f` does, a `fire-28-c` does not.

No options. Device required: yes, and it must be running with the USB system plugin.

Needs One ROM firmware v0.7.2 or later with the v0.2.2 or later USB system plugin. An older One ROM says so rather than reporting something invented.

inspect rgb

Show what the RGB LED is doing now — the mode, the colour, the brightness, how fast it is running, and which GPIO it is on.

```
onerom inspect rgb
```

On a `fire-28-c` running `onerom control rgb breathe --colour cyan --brightness 60 --period 4000`:

```
RGB LED:
  Mode:       breathe
  Colour:     #00FFFF (cyan)
  Brightness: 60%
  Period:     4000ms
```

A colour is named where it is one of the names `--colour` accepts. One that is not prints as hex alone — `#7F3C22`.

Each repeating mode has a shortest period it can run at, and a shorter one is refused rather than quietly run at the minimum:

Mode	Shortest period
`cycle`, `breathe`	1000ms
`flame`	500ms
`beacon`, `blink`	50ms

`cycle` walks the hues itself rather than showing a colour you set, so no `Colour` is reported while it runs:

```
RGB LED:
  Mode:       cycle
  Brightness: 25%
  Period:     3000ms
```

`--verbose` adds the GPIO:

```
RGB LED:
  Mode:       cycle
  Brightness: 25%
  Period:     3000ms
  GPIO:       44
```

Only some One ROM models have an RGB LED. On a board without one this reports:

```
RGB LED: this board does not have one
```

Where the RGB LED and the status LED share a GPIO — as they do on a `fire-24-f` — both commands report the same pin and say that it is shared. Both LEDs still work, and no mode is restricted.

No options. Device required: yes, and it must be running with the USB system plugin.

Needs One ROM firmware v0.7.2 or later with the v0.2.2 or later USB system plugin.

inspect header

Draw the connected device's pin (jumper / programming) header as ASCII. The board is inferred from the device. This is the device-oriented form of `board header`; see there for what the diagram shows.

```
onerom inspect header [--board <board>]
```

Option	Description
`--board`, `-b`	Board type, overriding what the connected One ROM reports. Only needed on a One ROM whose board type this build does not recognise.

`--board` is an override, not a substitute for the device: this command draws the board of a *connected* One ROM, so one must still be present. To draw a board by name with nothing connected, use `board header`.

inspect socket

Draw the connected device's ROM socket pinout as ASCII. The board is inferred from the device. This is the device-oriented form of `board socket`.

```
onerom inspect socket [--board <board>] [--chip-type <chip>] [--gpio]
```

Option	Description
`--board`, `-b`	Board type, overriding what the connected One ROM reports.
`--chip-type <chip>`, `-c`	Label pins with this ROM type's functions instead of GPIOs, and report the chip's image size on this board.
`--gpio`	Overlay the GPIO(s) onto the `--chip-type` function view (requires `--chip-type`).

As with `inspect header`, `--board` overrides the connected One ROM's reported board type rather than standing in for the device.

inspect otp

Print what One ROM's OTP contains:

- the board size
- the commissioning information
- the bootloader USB info

It doesn't check that the commissioning information is correctly signed. `hardware validate` does.

```
onerom inspect otp
onerom inspect otp --json
```

Option	Description
`--json`	Print the whole report as JSON instead of text.

`--verbose` adds:

- the external flash settings in `FLASH_DEVINFO`
- every commissioning instance and its entries, with their rows
- the bootloader settings rows and the page lock words

On a `fire-40-b` commissioned by Acme Retro:

```
$ onerom inspect otp
One ROM Fire 40 B - Firmware: v0.8.0 State: Stopped Serial: DE3F9C232F655B6B
  Board size: M
  Commissioned: yes
    Board type:   fire-40-b
    Manufacturer: Acme Retro
    Date:         2026-09-29
    Signing key:  Acme Retro (256)
  Bootloader USB info: set
    USB VID:               0x1209
    USB PID:               0xF540
    USB manufacturer:      piers.rocks
    USB product:           One ROM Bootloader
    Volume label:          ONEROM
    INDEX.HTM link:        https://onerom.org
    INDEX.HTM link name:   onerom.org
    INFO_UF2.TXT model:    One ROM
    INFO_UF2.TXT board ID: fire-40-b
```

© 2026 Piers Finlayson · onerom.org · MIT licence

monitor

Watch a running One ROM as it works.

```
onerom monitor <COMMAND>
```

Subcommand	Purpose	Device required
`log`	Show the One ROM's log as it is written	Yes

monitor log

Attach to the One ROM's USB serial port and print the firmware and plugin logging it sends, until the One ROM is disconnected, rebooted or stopped, or you press Ctrl-C.

Every attach opens with the One ROM naming itself, in a block headed `----- One ROM USB log -----`. What it has logged since anything last listened arrives after that, so attaching after a reboot still shows the boot log, which opens with a `-----` divider of its own:

```
Monitoring log - press Ctrl-C to stop
----- One ROM USB log -----
One ROM fire-32-b v0.8.0
Serial: 66CDC06786DCE3B4
Logging: boot, error, plugin-application
---------------------------
-----
One ROM v0.8.0.1 https://onerom.org
Copyright (c) 2026 Piers Finlayson <piers@piers.rocks>
Built: Oct  3 2026 18:48:06Z
Commit: 257953f
-----
RP235XB
RAM: 520KB
Flash: CS0 2048KB, CS1 0KB
Freq: 150MHz
```

`Name:` appears only when the One ROM has an instance name set, and `Logging:` lists the kinds of output switched on — see Logging. A One ROM that cannot forward its log says so there, rather than leaving you watching a silent port.

A One ROM plugin can claim exclusive use of the USB serial port. In this case, the CLI automatically retrieves logs using another mechanism.

```
onerom monitor log
onerom --serial 1234abcd monitor log
onerom monitor log --output boot.txt
```

Option	Description
`--output, -o <FILE>` (alias `--out`)	Also write the One ROM's output to this file, replacing its contents. The file receives what the One ROM sends and nothing else, so it is a transcript of the device rather than of this command. The output still appears on screen as well.

One ROM must be running for logging to be displayed.

Device required: yes.

console

Talk to the host system through One ROM's USB port.

```
onerom console [OPTIONS]
```

This command displays what One ROM sends, like `monitor log`, and additionally sends what you type to the retro system. Whether and how the retro system receives the input depends on the plugin(s) used.

By default you type a line, edit it, and press Enter to send it. `--raw` sends each key as you press it, for programs that read single keys. Ctrl-C exits and is never sent to the retro system.

```
onerom console
onerom console --raw
onerom console --line-ending crlf --output session.txt
echo 'LOAD "*",8' | onerom console
```

Option	Description
`--output, -o <FILE>` (alias `--out`)	Also write One ROM's output to this file, replacing its contents. The file contains only what One ROM sends. Output is still displayed.
`--line-ending <ENDING>`	What Enter sends: `cr`, `lf` or `crlf`. Default `cr`, which retro systems expect.
`--raw`	Send each key as you press it, not a line at a time. No line editing. Enter sends the line ending. Ctrl-C exits.
`--no-echo`	Don't display what you type. Use when the retro system echoes back what it receives. Requires `--raw`.
`--serial-port <PORT>`	Serial port to open: `1` or `2`. Default `1`. Port 1 is the USB plugin's default serial port. Port 2 requires the USB plugin and a user plugin that supports it.

If stdin is not a terminal, its contents are sent unchanged, and the command then keeps displaying output until One ROM disconnects or you press Ctrl-C. This can be used to send a file.

If the retro system is not reading the input pipe, typed input waits. Once One ROM's buffer is full, the command reports this on stderr after a second:

```
One ROM is not reading the input data
```

Input is sent, in order, once the retro system reads, and the command reports `Input resumed`.

Requirements:

- Firmware v0.7.3 or later.
- One ROM running, with both the USB and host-control plugins.
- For `--serial-port 2`, a user plugin that supports serial port 2.
- The notes under `monitor log` about the port, the banner, debug probes and the silence timeout also apply.

Device required: yes.

© 2026 Piers Finlayson · onerom.org · MIT licence

control

Transient actions on a connected One ROM. These affect current state but do not persist across power cycles.

```
onerom control <COMMAND>
```

Subcommand	Purpose	Device required
`reboot`	Reboot the device	Yes
`led`	Control the status LED	Yes
`standby`	Turn standby mode on or off	Yes (running)
`poke`	Write to SRAM or the live ROM image	Yes
`reset`	Pulse a GPIO low to reset the host system	Yes (running)
`select`	Select the active ROM slot **(not yet supported)**	Yes
`pin`	Drive a pin high, low or high-impedance	Yes (running)
`erase`	Erase flash memory	Yes

control reboot

Restart the firmware; the device re-initialises and resumes serving. By default pauses afterwards for re-enumeration. Also reachable as the top-level `reboot`.

```
onerom control reboot
```

Option	Description
`--stopped, -p`	Reboot into stopped (bootloader) state.
`--running, -r`	Reboot into running (serving) state. Default.
`--fast`	Don't pause for re-enumeration.
`--msd, -m`	Mount mass storage when rebooting into stopped mode. Conflicts with `--running`.

`--stopped` and `--running` are mutually exclusive.

control led

Control the status LED — the single-colour LED every One ROM has. The RGB LED that some models support is driven by `control rgb` instead.

```
onerom control led on
onerom control led off
onerom control led beacon --hold 10000
onerom control led flame --period 1200
onerom control led blink
```

© 2026 Piers Finlayson · onerom.org · MIT licence

Subcommand	Description
`on`	Turn the status LED on.
`off`	Turn the status LED off.
`beacon`	Beacon the LED to identify a physical unit.
`flame`	Flame effect on the LED.
`blink`	Blink the LED on and off until something changes it.

Option	Description	Subcommands
`--hold <MS>`	Stay in this mode for this many milliseconds, then go back to what the LED was doing before. The device times it, so it completes even if the command does not. Maximum 60000.	all
`--period <MS>`	Milliseconds for one repetition — one blink for `beacon` and `blink`, one pass of the flicker for `flame`. Defaults to 100, 1000 and 575 respectively. Minimum 50 for `beacon` and `blink`, 500 for `flame`.	`beacon`, `blink`, `flame`

`beacon` ends by itself after 2500ms unless `--hold` says otherwise. `blink` is the same on-and-off toggle but slower and unbounded — it runs until something changes it, or until a `--hold` you give it expires.

The status LED is lit or dark, so it takes no colour and no brightness. `cycle` and `breathe` are built out of a colour and are the two modes it cannot do.

`--hold` and `--period` need One ROM firmware v0.7.2 or later with the v0.2.2 or later USB system plugin. The CLI checks before sending, and says so rather than reporting success on a device that would ignore them. A plain `on`, `off`, `beacon` or `flame` works on any One ROM and costs no extra exchange with the device.

Device required: yes.

control rgb

Control the RGB LED that some One ROM models carry. For the single-colour status LED every model has, see `control led`.

```
onerom control rgb on --colour red
onerom control rgb on --colour #FF8000 --brightness 40
onerom control rgb cycle --period 3000
onerom control rgb off
```

Subcommand	Description
`on`	Light the LED at a colour.
`off`	Turn the LED off.
`beacon`	Beacon the LED to identify a physical unit. Ends by itself after 2500ms.
`flame`	Flame effect on the LED.
`cycle`	Rotate through the hues.

© 2026 Piers Finlayson · onerom.org · MIT licence

Subcommand	Description
`breathe`	Fade the colour up and down.
`blink`	Alternate the colour with dark.

Option	Description	Subcommands
`--colour <COLOUR>` (alias `--color`)	A name, or hex written `#RRGGBB` or `0xRRGGBB`. Defaults to red.	all but `off` and `cycle`
`--brightness <PERCENT>`	1 to 100. Omit for the device's default, which is deliberately modest — an RGB LED at full output is uncomfortable at desk distance.	all but `off`
`--period <MS>`	Milliseconds for one repetition.	`beacon`, `flame`, `cycle`, `breathe`, `blink`
`--hold <MS>`	Stay in this mode for this many milliseconds, then go back to what the LED was doing before. The device times it, so it completes even if the command does not. Maximum 60000.	all

The named colours are `red`, `green`, `blue`, `white`, `yellow`, `cyan`, `magenta`, `orange`, `purple` and `pink`.

`cycle` chooses its own colours, so it takes no `--colour`.

Each repeating mode has a shortest period it can run at, and a shorter one is refused rather than quietly run at the minimum:

Mode	Default period	Shortest period
`cycle`, `breathe`	5000ms	1000ms
`flame`	575ms	500ms
`blink`	1000ms	50ms
`beacon`	100ms	50ms

Nothing is printed unless the CLI is verbose:

```
$ onerom --verbose control rgb on --colour orange --brightness 40
RGB LED on
```

Read back what the LED is doing with `inspect rgb`:

```
RGB LED:
  Mode:       on
  Colour:     #FF6000 (orange)
  Brightness: 40%
```

Only some One ROM models have an RGB LED. On a board without one, these commands say so rather than appearing to work. Needs One ROM firmware v0.7.2 or later with the v0.2.2 or later USB system plugin.

Device required: yes, and it must be running with the USB system plugin.

© 2026 Piers Finlayson · onerom.org · MIT licence

control standby

Turn standby mode on or off. In standby mode One ROM doesn't serve the ROM but is listening on the bus.

The data pins remain undriven, and plugins run as normal. Turning standby off starts serving the selected ROM. A slot boots in standby when `standby=on` is provided in its `--slot` specification, or with `fire.standby` set in its chip set's `firmware_overrides`.

```
onerom control standby on
onerom control standby off
```

Subcommand	Description
`on`	Turn standby mode on.
`off`	Turn standby mode off.

Success prints nothing. With `--verbose` it prints `Standby on` or `Standby off`.

The device must be running with the USB system plugin and firmware v0.8.0 or later. With an older plugin or firmware the command fails:

```
$ onerom control standby on
Failed to execute command.
This One ROM's USB system plugin predates standby.
  One ROM Fire 24 F - Firmware: v0.8.0 State: Running Serial: DE3F9C232F655B6B
  Reprogram it with the v0.8.0 or later USB system plugin, for example:
    onerom program --config <CONFIG> --plugin usb
```

`inspect gpio`, `inspect slots`, `peek live` and `poke live` mark their output `standby` while One ROM is in standby.

control poke

Transient writes to device memory — changes are lost on reboot. Use `update` for persistent flash writes.

```
onerom control poke <COMMAND>
```

control poke memory

Write a single byte or a binary file to SRAM at a provided address. When the device is running, virtual addresses are available (e.g. `0x90000000` is the start of the live ROM image — though prefer `poke live` for that). Writing arbitrary SRAM can corrupt firmware state.

```
onerom control poke memory --address 0x20000010 --byte 0xFF
onerom control poke memory --address 0x20000000 --input patch.bin
```

Option	Description
`--address, -a <ADDRESS>` (alias `--addr`)	Address to write to. Decimal or `0x` hex.
`--byte <BYTE>` (alias `--value`)	Single byte value to write. Decimal or hex.
`--input, -i <FILE>` (alias `--in`)	Write the contents of this binary file.

© 2026 Piers Finlayson · onerom.org · MIT licence

Exactly one of `--byte` / `--input` is required.

control poke live

Write a single byte or a binary file to the live ROM image at a **logical** ROM offset (starting at 0). Useful for patching a running ROM without reflashing. Also reachable as the top-level `poke`.

```
onerom control poke live --address 0x100 --byte 0xEA
onerom control poke live --address 0 --input patch.bin
```

Option	Description
`--address, -a <ADDRESS>` (alias `--addr`)	Logical ROM address, starting at 0. Decimal or `0x` hex. Default `0`. On a One ROM serving half a 27C080, address 0 is the start of that half.
`--byte <BYTE>` (alias `--value`)	Single byte value to write. Decimal or hex.
`--input, -i <FILE>` (alias `--in`)	Write the contents of this binary file.
`--delta` (alias `--deltas`)	Only write bytes that differ from current device content. Requires `--input`.
`--dry-run` (alias `--dryrun`)	Show what would be written without writing. Requires `--delta`.

Exactly one of `--byte` / `--input` is required.

In standby the line ends `(standby)`:

```
$ onerom control poke live --address 0x100 --byte 0xea
Wrote 1 byte(s) to live ROM offset 0x00000100 (standby)
```

control reset

Pulse a GPIO low, then release it, to reset the host system One ROM is installed in — useful in scripted workflows after programming a new image. `program --reset-host` does the same thing as the last step of programming, and is the shorter way to say it.

`--pin` is the pin connected to the host's reset line, typically `X1`, `X2` or an image select pin, for example `sel_c`. Write it as a header pin (`sel_c`, `x1`) or as an MCU GPIO (`gpio9`) — see Pin values. `inspect header` lists each header pin's GPIO. Reserve the pin when the image is built — see Reserve a pin for another use.

The line is only ever **driven low and then released to high impedance**. A reset net has its own pull-up and may have other drivers on it, so there is deliberately no way to drive it high. Use `control pin` if you need arbitrary states.

The **device** times the pulse, not the CLI: if this command is interrupted, the terminal closes or the cable is pulled mid-pulse, the device still releases the pin. The device's own limit is 60 seconds.

The device must be **running** with the USB system plugin — see Device states.

```
onerom control reset --pin sel_c
onerom control reset --pin gpio9
onerom control reset --pin gpio9 --hold 500
```

© 2026 Piers Finlayson · onerom.org · MIT licence

Option	Description
`--pin <PIN>`	Pin the reset wire is connected to, written as `gpio<N>` or as a header pin (see Pin values). Required.
`--board <BOARD>`	Board type, overriding what the device reports. Only needed to resolve a `--pin` header pin on a board this build does not recognise.
`--hold <MS>`	Milliseconds to hold reset asserted. Decimal or `0x` hex. Default `100`; `0` is rejected, because a reset pulse with no end is not a reset.

If One ROM is itself using the GPIO the command is refused, naming what it is doing; `control reset` has no `--force` of its own, and the message points at `control pin --force` for the case where that is genuinely what you want. If the GPIO is not 5V-tolerant the command warns and asks for confirmation, which `--yes` answers.

The pulse counts toward the device's limit on pins under a timed hold at once — see `control pin`.

```
$ onerom control reset --pin x1
Asserted reset on x1 (gpio9) for 100ms - the device times the pulse and releases the pin
```

control select

Switch the device to serving the specified slot immediately (not persistent). **(not yet supported)**

Option	Description
`--slot <INDEX>`	Slot index to activate. Required.

control pin

Drive a One ROM pin high, low or high-impedance, optionally for a bounded period.

`--pin` is an MCU GPIO or a header pin (see Pin values).

Without `--hold` the state is latched until something else changes it. With `--hold` the **device** holds the state for that many milliseconds and then applies `--then` — high impedance unless you say otherwise. As with `control reset`, the hold is timed on the device, so an interrupted CLI cannot leave a pin latched.

A limited number of pins can have a timed `--hold` at once. A pin driven without `--hold` is latched indefinitely and is not included in that limit. One ROM rejects a hold on a further pin once the limit is reached.

The device must be **running** with the USB system plugin — see Device states. `inspect gpio` shows what each GPIO is and what One ROM is using it for.

```
onerom control pin --pin gpio9 --state high
onerom control pin --pin gpio9 --state low --hold 250
onerom control pin --pin gpio9 --state low --hold 250 --then high
onerom control pin --pin x1 --state 0 --hold 250
onerom control pin --pin sel_a --state z
```

Option	Description
`--pin <PIN>`	Pin to drive, written as `gpio<N>` or as a header pin (see Pin values). Required.

© 2026 Piers Finlayson · onerom.org · MIT licence

Option	Description
`--board <BOARD>`	Board type, overriding what the device reports. Only needed to resolve a `--pin` header pin on a board this build does not recognise.
`--state <STATE>`	`high`, `low`, or `z` (high-impedance). `1` and `0` are accepted for `high` and `low`. Required.
`--hold <MS>`	Hold `--state` for this many milliseconds, then apply `--then`. Decimal or `0x` hex. Omit to latch indefinitely. The device's own limit is 60 seconds.
`--then <STATE>`	State to apply when `--hold` expires: `high`, `low` or `z` (or `1`/`0`). Default `z`. Requires `--hold`.
`--force`	Drive the GPIO even though One ROM or a plugin is using it.

Refusals and warnings. If One ROM or a plugin is using the GPIO, the command fails and displays what the GPIO is used for. `--force` can be used to override it.

If the GPIO is not 5V-tolerant — an RP2350 ADC pin, per the board metadata, not a measurement — the command warns and asks for confirmation, which `--yes` or `--force` answers. Nothing else about the pin is checked: what is wired to it, whether a jumper is fitted and what voltage the far end sits at are yours to know.

```
$ onerom control pin --pin x1 --state low --hold 2000
Set x1 (gpio9) low for 2000ms - the device times the hold and then sets it high impedance
```

control erase

Permanently erase flash contents — firmware, metadata and ROM images. A fully erased unit boots into One ROM's bootloader and is reprogrammed with `--unrecognised` + `--board`.

Best performed while stopped; by default the command reboots into the required state first. Erasing the core firmware or the system plugin while **running** takes down the USB stack (requiring manual BOOTSEL), and large erases may cause a temporary USB drop and re-enumerate — in which case the erase likely succeeded and can be checked with `inspect peek memory`. Anything else running from flash (e.g. a user plugin) may crash during an erase.

Offsets are relative to the flash base `0x10000000`. Ranges must be 4096-aligned. Multiple ranges may be erased in one operation.

```
onerom control erase --all
onerom control erase --offset 0x20000 --length 0x1000
```

Option	Description
`--all, -a`	Erase all flash contents.
`--offset <OFFSET>`	Erase at offset(s) from the flash base. 4096-aligned; pair each with a `--length`; repeatable. Conflicts with `--address`.
`--address <ADDRESS>` (alias `--addr`)	Erase at absolute address(es). 4096-aligned; pair each with a `--length`; repeatable. Conflicts with `--offset`.
`--length <LENGTH>` (aliases `--len`, `--size`)	Length of each range. 4096-aligned; specify once per `--offset`/`--address`; repeatable. Conflicts with `--all`.

© 2026 Piers Finlayson · onerom.org · MIT licence

Option	Description
`--no-reboot, -n`	Don't reboot before or after erasing. Risky if One ROM is accessing the range.
`--stopped, -p`	Reboot into stopped mode after erasing.
`--running, -r`	Reboot into running mode after erasing.
`--msd, -m`	Mount mass storage when rebooting into stopped mode. Requires `--stopped`.
`--fast`	Don't pause for re-enumeration. Requires a reboot mode.

One of `--all` / `--offset` / `--address` is required. `--stopped` and `--running` are mutually exclusive, and both conflict with `--no-reboot`.

© 2026 Piers Finlayson · onerom.org · MIT licence

update

Make persistent modifications to a connected One ROM.

Each change is written to the One ROM's flash so it survives power cycles.

```
onerom update <COMMAND>
```

Subcommand	Purpose	Device required
`firmware`	Change a One ROM's firmware version, keeping its configuration and ROM images	Yes
`slot`	Replace a ROM image in a slot, keeping the configuration and the other ROM images	Yes
`plugin`	Add or replace a plugin keeping the rest of the configuration	Yes
`commit`	Commit the live ROM image to flash **(not yet supported)**	Yes

update firmware

Change a One ROM's firmware version, keeping its configuration and ROM images.

Each published plugin is updated to its latest compatible release and any stored plugin data is reset. An unpublished plugin is left as-is.

```
onerom update firmware
onerom update firmware --version 0.8.1
```

Example output:

```
$ onerom update firmware --version 0.8.1
Updating firmware from v0.8.0 to v0.8.1
Updating plugin usb from v0.3.1 to v0.3.2
Programming device - DO NOT DISCONNECT
Programming complete
```

Option	Description
`--version <VERSION>`	Firmware version to change to. Defaults to the latest release. Conflicts with `--base-firmware`.
`--base-firmware <FILE>`	Local base firmware to build with instead of downloading a release.
`--force, -f`	Continue despite non-fatal problems. Each problem is printed as a warning.

update slot

Replace a ROM image in a slot, keeping the configuration and the other ROM images.

The new ROM image has the same chip type and chip-select settings as the one it's replacing, and its filename/label comes from `--image`.

© 2026 Piers Finlayson · onerom.org · MIT licence

The firmware version and plugins are unchanged.

```
onerom update slot --slot 2 --image kernal.bin
onerom update slot --slot 3 --chip 1 --image char.bin
```

Option	Description
`--slot <INDEX>`	Slot being updated. For slot numbering see `onerom inspect slots`. Required.
`--chip <INDEX>`	Chip to replace in a slot with multiple chips. Chips are numbered from 0.
`--image <FILE>`	ROM image file or http(s) URL. Required.
`--force, -f`	Continue despite non-fatal problems. Each problem is printed as a warning.

update plugin

Add or replace a plugin keeping the rest of the configuration.

A new plugin replaces the plugin of the same type. A user plugin requires a system plugin be present. The other plugin is unchanged, including any data it has stored.

```
onerom update plugin --plugin usb
onerom update plugin --plugin host-control
```

Option	Description
`--plugin <SPEC>`	Plugin to add. See Plugin specification. Required.
`--force, -f`	Continue despite non-fatal problems. Each problem is printed as a warning.

update commit

Commit the live ROM image to flash. **(not yet supported)**

The live ROM image is written to its slot in flash so it survives power cycles.

```
onerom update commit
onerom update commit --slot 2
```

Option	Description
`--slot <INDEX>`	Slot to commit. Defaults to the active slot.

© 2026 Piers Finlayson · onerom.org · MIT licence

clone

Copy a One ROM or firmware file to another One ROM.

The source can be a One ROM or a firmware file.

The destination can be a different board type but must:

- support the source's chip types
- have any pins the source's config reserves
- have sufficient flash for the source's ROM images

The source One ROM's firmware version and plugins are copied over but the destination retains its own instance name and USB serial override (if any).

```
onerom clone --from E126C9F97C10ADAC --to FC9D67248E8E8023
onerom clone --from old.bin --to FC9D67248E8E8023
```

Option	Description
`--from <SOURCE>`	Source One ROM to copy from. Can be the serial number of the One ROM to copy or a firmware file. Required.
`--to <DEVICE>`	Serial number of the destination One ROM. Required.
`--board, -b <BOARD>`	Board type of the destination One ROM. Required for an uncommissioned One ROM without One ROM firmware.
`--force, -f`	Continue despite non-fatal problems. Each problem is printed as a warning.

Device required: yes, and a source One ROM unless `--from` is a firmware file.

© 2026 Piers Finlayson · onerom.org · MIT licence

hardware

Commissioning sets hardware properties in the RP2350's OTP (One Time Programmable) memory, which cannot be erased. COMMISSIONING describes the process.

`commission`, `request-signature` and `set-size` require an RP2350 of stepping A3 or A4, as One ROM requires one of these steppings.

```
onerom hardware <COMMAND>
```

Subcommand	Purpose	Device required
`commission`	Commission a One ROM	Yes
`request-signature`	Request a signature to commission a One ROM	Yes
`sign`	Sign a One ROM's commissioning instance	No
`set-size`	Set a One ROM's size	Yes
`validate`	Check a One ROM's commissioning information	Yes

Each command other than `sign`:

- stops a One ROM that is running or in limp mode first, and reboots it into running mode once complete
- requires `--unrecognised` for an uncommissioned One ROM without One ROM firmware

`commission`, `sign` and `set-size` list what they will write or sign and ask for confirmation, which `--yes` answers.

`commission` and `set-size`:

- reboot a One ROM that was already stopped back into stopped mode after writing OTP
- complete an interrupted run when run again

Board sizes

`--size` takes:

- `M`, with 2MB of flash
- `L`, with 4MB of flash

A board that doesn't support external flash is always M.

Setting a One ROM to a size other than M is **permanent** so it is recommended to test the external flash first. A One ROM's size can be changed from M with `hardware set-size`. OTP describes how each size is recorded.

hardware commission

Write to the RP2350's OTP:

- the signed board details
- the USB info that white labels the RP2350's bootloader as One ROM

- the external flash settings for a size other than M

```
onerom hardware commission --board fire-40-b --size M --manufacturer "Acme Retro" --key acme.pem
onerom hardware commission --board fire-24-f --manufacturer "Acme Retro" --key acme.pem --dry-run
onerom hardware commission --board fire-40-b --size L --manufacturer "Acme Retro" \
    --signer https://sign.internal.example.com --key-id 256
onerom hardware commission --board fire-40-b --size M --manufacturer onerom.org \
    --date 20260929 --key-id 2 --signature 3f0c...
onerom hardware commission --board fire-40-b --size M --manufacturer "Acme Retro" --key acme.pem
\
    --validate --inspect-otp
```

Option	Description
`--board, -b <BOARD>`	Board type, for example `fire-40-b`. Must be a Fire board with the matching RP2350 package (A or B). Required.
`--size <SIZE>`	The board's size, for example `M`. Required for a board that supports external flash.
`--manufacturer <NAME>`	Manufacturer's name, for example `"Acme Retro"`. 1 to 32 printable ASCII characters without `*` or a leading or trailing space. Required.
`--revision <REVISION>` (alias `--rev`)	The board's revision, for example `A1`. 1 to 16 printable ASCII characters without a leading or trailing space.
`--batch <BATCH>`	The board's manufacturing batch, for example a PCB order number. 1 to 16 printable ASCII characters without a leading or trailing space.
`--signer <URL>`	Signing server's address, for example `https://sign.internal.example.com`. Must use https. Requires `--key-id`.
`--key-id <ID>`	Signing key's ID from 1 to 65535, for example `256`. Conflicts with `--key`.
`--pin <PIN>`	PIN of the signing server's key or of an encrypted key file. Asked for in a terminal if omitted. Conflicts with `--signature`.
`--key <FILE>`	Signing key file containing an Ed25519 private key in PKCS#8 PEM form, for example `acme.pem`.
`--signature <SIGNATURE>`	Commissioning signature as 128 hex digits, generated by `hardware sign`. Requires `--key-id` and `--date`.
`--date <DATE>`	Commissioning date as `YYYYMMDD`, for example `20260929`. Defaults to today's UTC date.
`--force, -f`	Allow the cases listed below.
`--dry-run` (alias `--dryrun`)	List what would be written without writing it.
`--validate`	After commissioning, check the commissioning information as `hardware validate` does. Conflicts with `--dry-run`.
`--inspect-otp`	After commissioning, show the contents of the OTP as `inspect otp` does. Runs after `--validate`. Conflicts with `--dry-run`.

The signature comes from one of:

- a key file, with `--key` — see Using a Signing Key
- a signing server, with `--signer` and `--key-id` — see Signing Server
- a signing request or `hardware sign`, with `--signature`, `--key-id`, `--date` and any `--revision` and `--batch` it was signed with

The signing key must be authorised for the manufacturer being signed.

`--force` is required to:

- re-commission a One ROM with different details — see Correcting Commissioning Errors
- commission a One ROM whose firmware is for another board type
- commission a One ROM as M when another size is partly programmed
- use a `--date` in the future

`--verbose` adds the bootloader USB info and each row with its value.

Check the result with `--validate`, or afterwards with `hardware validate`. `--validate` and `--inspect-otp` run after OTP has been written and the One ROM rebooted into stopped mode, so the settings take effect.

Commissioning a `fire-40-b` as M with an encrypted key file:

```
$ onerom hardware commission --board fire-40-b --size M --manufacturer "Acme Retro" --key
acme.pem
PIN for acme.pem:
One ROM Fire 40 B - Firmware: v0.8.0 State: Stopped Serial: DE3F9C232F655B6B
Commissioning:
  Board type:   fire-40-b
  Board size:   M
  Manufacturer: Acme Retro
  Date:         2026-09-29
  Signing key:  Acme Retro (256)
To write:
  Commissioning instance at row 0x0c0   62 rows
  Lock for page 3                        1 row
  Bootloader USB strings                41 rows
  White label table                      9 rows
  USB_WHITE_LABEL_ADDR                   1 row
  USB_BOOT_FLAGS and its copies          3 rows
  Total                                117 rows
WARNING: OTP writes cannot be undone.
Write 117 rows? (y/N): y
Writing OTP - DO NOT DISCONNECT
Commissioning instance: written
Lock for page 3: written
Bootloader USB strings: written
White label table: written
USB_WHITE_LABEL_ADDR: written
USB_BOOT_FLAGS and its copies: written
Commissioning complete
```

hardware request-signature

Generate a signing request for the One ROM. The maintainers answer it with the `hardware commission` command to run. See Submitting a Signing Request.

A One ROM commissioned this way is signed with the piers.rocks Community signing key and its manufacturer is always `onerom.org`.

```
onerom hardware request-signature --board fire-40-b --size M
onerom hardware request-signature --board fire-24-f
```

© 2026 Piers Finlayson · onerom.org · MIT licence

Option	Description
`--board, -b <BOARD>`	Board type, for example `fire-40-b`. Must be a Fire board with the matching RP2350 package (A or B). Required.
`--size <SIZE>`	The board's size, for example `M`. Required for a board that supports external flash.

Requesting a signature for a `fire-40-b` as M:

```
$ onerom hardware request-signature --board fire-40-b --size M
One ROM Fire 40 B - Firmware: v0.8.0 State: Stopped Serial: DE3F9C232F655B6B
Signing request:
  Chip ID:      DE3F9C232F655B6B
  Board type:   fire-40-b
  Board size:   M
  Manufacturer: onerom.org
Open this link to raise a signing request for this One ROM on GitHub:
  https://github.com/piersfinlayson/one-rom/issues/new?template=signing-
request.yml&title=%5Bsigning%20request%5D%20DE3F9C232F655B6B&chip_id=DE3F9C232F655B6B&board=fire-
40-b&size=M&manufacturer=onerom.org&cli_version=0.6.0
```

hardware sign

Sign a One ROM's commissioning information by its Chip ID. It prints the `hardware commission` command to run on that One ROM.

A signing server publicly records the signature once confirmed.

```
onerom hardware sign --chip-id DE3F9C232F655B6B --board fire-40-b --size M \
    --manufacturer "Acme Retro" --key acme.pem
onerom hardware sign --chip-id DE3F9C232F655B6B --board fire-24-f \
    --manufacturer "Acme Retro" --key acme.pem --json
onerom hardware sign --chip-id DE3F9C232F655B6B --board fire-40-b --size L \
    --manufacturer "Acme Retro" --signer https://sign.internal.example.com --key-id 256 --dry-run
```

Option	Description
`--chip-id <CHIPID>`	One ROM's Chip ID as 16 hex digits, for example `DE3F9C232F655B6B`. `scan --verbose` prints it. Required.
`--board, -b <BOARD>`	Board type, for example `fire-40-b`. Must be a Fire board. Required.
`--size <SIZE>`	The board's size, for example `M`. Required for a board that supports external flash.
`--manufacturer <NAME>`	Manufacturer's name, for example `"Acme Retro"`. 1 to 32 printable ASCII characters without `*` or a leading or trailing space. Required.
`--revision <REVISION>` (alias `--rev`)	The board's revision, for example `A1`. 1 to 16 printable ASCII characters without a leading or trailing space.
`--batch <BATCH>`	The board's manufacturing batch, for example a PCB order number. 1 to 16 printable ASCII characters without a leading or trailing space.
`--date <DATE>`	Commissioning date as `YYYYMMDD`, for example `20260929`. Defaults to today's UTC date. Cannot be in the future.
`--signer <URL>`	Signing server's address, for example `https://sign.internal.example.com`. Must use https. Requires `--key-id`.

© 2026 Piers Finlayson · onerom.org · MIT licence

Option	Description
`--key-id <ID>`	Signing key's ID from 1 to 65535, for example `256`. Conflicts with `--key`.
`--pin <PIN>`	PIN of the signing server's key or of an encrypted key file. Asked for in a terminal if omitted.
`--key <FILE>`	Signing key file containing an Ed25519 private key in PKCS#8 PEM form, for example `acme.pem`.
`--dry-run` (alias `--dryrun`)	Sign without asking for confirmation and without the signing server publicly recording the signature. Doesn't print the `hardware commission` command. Conflicts with `--key`.
`--json`	Print the result as JSON instead of text.

The signature comes from one of:

- a key file, with `--key` — see Using a Signing Key
- a signing server, with `--signer` and `--key-id` — see Signing Server

The signing key must be authorised for the manufacturer being signed.

Signing for a `fire-40-b` as M with an encrypted key file:

```
$ onerom hardware sign --chip-id DE3F9C232F655B6B --board fire-40-b --size M --manufacturer "Acme
Retro" --key acme.pem
PIN for acme.pem:
Signing:
  Chip ID:      DE3F9C232F655B6B
  Board type:   fire-40-b
  Board size:   M
  Manufacturer: Acme Retro
  Date:         2026-09-29
  Signing key:  Acme Retro (256)
Sign? (y/N): y
Signature:
ff0179e1256e4cd83c69939e9dc52182d2f4d974d532c227c10226038d3ba7504521496c649884133c86445b512fce27e
80bcc1118668354f2885e02d1c80a0c
Command to commission the One ROM:
  onerom hardware commission --board fire-40-b --size M --manufacturer 'Acme Retro' --date
20260929 --key-id 256 --signature
ff0179e1256e4cd83c69939e9dc52182d2f4d974d532c227c10226038d3ba7504521496c649884133c86445b512fce27e
80bcc1118668354f2885e02d1c80a0c
```

hardware set-size

Write a One ROM's size settings to OTP without commissioning it. `hardware commission` writes the same settings, so this is only required when changing a One ROM's size after or without commissioning. Size M writes nothing.

```
onerom hardware set-size --board fire-40-b --size L
onerom hardware set-size --board fire-40-b --size L --dry-run
```

Option	Description
`--board, -b <BOARD>`	Board type, for example `fire-40-b`. Must be a Fire board with the matching RP2350 package (A or B), even with `--force`. Required.
`--size <SIZE>`	The board's size, for example `L`. Required.

Option	Description
`--force, -f`	Set the size of a One ROM whose firmware is for another board type.
`--dry-run` (alias `--dryrun`)	List what would be written without writing it.

On a commissioned One ROM `--board` must be the board type it is commissioned as.

`--verbose` adds each row with its value.

Changing a `fire-40-b` commissioned as M to L:

```
$ onerom hardware set-size --board fire-40-b --size L
One ROM Fire 40 B - Firmware: v0.8.0 State: Stopped Serial: DE3F9C232F655B6B
Setting board size:
  Board type: fire-40-b
  Board size: L
To write:
  FLASH_DEVINFO                                    1 row
  FLASH_DEVINFO_ENABLE in BOOT_FLAGS0 and its copies  3 rows
  Total                                            4 rows
WARNING: OTP writes cannot be undone.
Write 4 rows? (y/N): y
Writing OTP - DO NOT DISCONNECT
FLASH_DEVINFO: written
FLASH_DEVINFO_ENABLE in BOOT_FLAGS0 and its copies: written
Board size set
```

hardware validate

Check the signature of each commissioning instance on the One ROM against the authorised signing keys. The command fails unless the One ROM's latest commissioning instance is valid.

```
onerom hardware validate
onerom hardware validate --json
```

Option	Description
`--json`	Print the result as JSON instead of text.

The command downloads the most recent signing key table. Where it cannot, it uses the table built into the CLI and prints a warning.

`--verbose` adds:

- the signing key table's source
- the row of the commissioning instance in use
- every other commissioning instance

Validating a `fire-40-b` commissioned by Acme Retro:

© 2026 Piers Finlayson · onerom.org · MIT licence

```
$ onerom hardware validate
One ROM Fire 40 B - Firmware: v0.8.0 State: Stopped Serial: DE3F9C232F655B6B
Board size: M
Commissioned: yes
  Board type:   fire-40-b
  Manufacturer: Acme Retro
  Contact:      https://github.com/acme-retro
  Date:         2026-09-29
  Signing key:  Acme Retro (256)
  Signature:    verified
Commissioning information valid
```

image

ROM image file manipulation. No device connection required.

```
onerom image <COMMAND>
```

image swap-bytes

Swap adjacent byte pairs — reverses byte order within each 16-bit word throughout the image. Required for 16-bit ROM types (e.g. 27C400) when the source has the opposite byte order to what One ROM expects. The input must have an even number of bytes.

```
onerom image swap-bytes --input kick.bin --output kick-swapped.bin
```

Option	Description
`--input, -i <FILE>` (alias `--in`)	Input ROM image file.
`--output, -o <FILE>` (alias `--out`)	Output file path.

The same operation is available during a build as `--slot transform=swap_bytes`; see Image transforms.

Before writing, the input is checked against a list of known 16-bit ROM headers. Where it is recognised and swapping it would be incorrect to program with One ROM, a warning is printed and the swap still goes ahead:

```
$ onerom image swap-bytes --input kick-swapped.bin --output out.bin
Warning: kick-swapped.bin starts with an Amiga ROM header, low byte of each pair first.
  It is already the way One ROM needs it, and swapping the bytes will stop it
  working with One ROM.
Written to out.bin
```

See 16-bit ROM image byte ordering.

Device required: no.

image deinterleave

Extract one lane from an interleaved ROM image. The image contains `--stride` interleaved lanes of `--bytes` bytes each; lane `--offset` is kept and the rest discarded. Used to split a wide ROM image, distributed as a single interleaved file, into the narrower images each device needs.

The input length must be a multiple of `--bytes × --stride`; the output is `1/--stride` of the input length.

© 2026 Piers Finlayson · onerom.org · MIT licence

```
# odd bytes of a 16-bit interleaved image
onerom image deinterleave --input rom16.bin --output odd.bin --offset 1 --stride 2

# byte 2 of a 32-bit interleaved image
onerom image deinterleave --input rom32.bin --output b2.bin --offset 2 --stride 4

# the upper 16-bit half of each 32-bit word
onerom image deinterleave --input rom32.bin --output hi.bin --offset 1 --stride 2 --bytes 2
```

Option	Description
`--input, -i <FILE>` (alias `--in`)	Input ROM image file.
`--output, -o <FILE>` (alias `--out`)	Output file path.
`--offset <N>`	Which lane to keep. Must be less than `--stride`.
`--stride <N>`	How many lanes the image interleaves. Must be at least 2.
`--bytes <N>` (alias `--unit`)	Width of one lane, in bytes. Defaults to `1`; use `2` to keep 16-bit words together.

The same operation is available during a build as `--slot transform=deinterleave:<offset>/<stride>[/<bytes>]` ; see Image transforms.

Device required: no.

image convert

Convert a ROM image between formats. Reads `--input` in the `--from` format and writes `--output` in the `--to` format. Formats: `binary` (aliases `bin` , `raw`), `ihex` (Intel HEX; aliases `intel-hex` , `intel_hex`) and `srec` (Motorola S-record; aliases `s-record` , `s_record` , `srecord` , `motorola` , `s19`). Any format can be converted to any other, including `ihex` to `srec` . The format set is designed to grow — further formats can be added without changing the command.

```
onerom image convert --from ihex --to binary --input rom.hex --output rom.bin
onerom image convert --from binary --to srec --input rom.bin --output rom.s19 --load-address
$E000
```

Option	Description
`--from <FORMAT>`	Input format: `binary`, `ihex` or `srec`.
`--to <FORMAT>`	Output format: `binary`, `ihex` or `srec`.
`--input, -i <FILE>` (alias `--in`)	Input ROM image file.
`--output, -o <FILE>` (alias `--out`)	Output file path.
`--load-address <ADDR>`	Load address (decimal, or `0x`/`$`-prefixed hex). Only valid when one side is `ihex` or `srec`; subtracted when reading that format, used as the base when writing it. Defaults to `0`.

Both record formats are written with 16-byte data records and a terminating record; unwritten addresses read as `0xFF` when decoding. S-record output uses one data record type throughout, the narrowest that addresses the whole image — `S1` below 64 KB, then `S2` , then `S3` — with the paired `S9` / `S8` / `S7`

terminator. An S-record file that ends without a termination record is read, which is what `srec_cat` writes unless it is given an execution start address. Device required: no.

firmware

Build, inspect and manage firmware binaries. Use `program` to flash; `firmware build` produces a binary without flashing.

```
onerom firmware <COMMAND>
```

Subcommand	Purpose	Device required
`build`	Build a firmware binary from a config	No
`inspect`	Inspect a firmware binary	No
`releases`	List firmware releases	No
`download`	Download a release binary	No
`chips`	List supported chip types and their flash usage	No
`program`	Alias for `onerom program`	Yes

firmware build

Produce a flashable firmware binary for a board and MCU from a JSON config or inline `--slot` args, without flashing.

```
onerom firmware build --config c64.json --board fire-24-e --out firmware.bin
onerom firmware build --board fire-24-e \
    --slot file=kernal.bin,type=2364,cs1=active-low \
    --out firmware.bin
onerom firmware build --config amiga.json --board fire-40-b --size L --out firmware.bin
```

The configuration options mirror `program`: `--config` (`-j`), `--slot`, `--plugin`, `--no-plugin`, `--config-name`, `--config-description`, `--save-config`, `--no-config`, and the per-device overrides `--instance-name`, `--serial-override`, `--logging`, `--disable-swd`, `--turbo-boot`, `--reserve-pin` and `--addr-mon-min-cs-act-ns`. `--no-config` can't be used with `--config`, `--slot` or a per-device override. Build-specific options:

Option	Description
`--board, -b <BOARD>`	Target board type. Required when not inferrable from a connected device.
`--size <SIZE>`	The board's size, for example `L`. Defaults to `M`. A size other than M requires firmware v0.8.0 or later.
`--version <VERSION>`	Firmware version to build against. Defaults to latest.
`--base-firmware <FILE>`	Use a local minimal firmware instead of downloading. Conflicts with `--version`.
`--output, -o <FILE>` (alias `--out`)	Output file path. Defaults to `onerom_<board>_v<version>_<config>.bin`, where `<config>` is the `--config` filename without its extension. Conflicts with `--path`.

Option	Description
`--path <DIR>`	Output directory, using the default filename. Conflicts with `--output`.
`--force, -f`	Continue despite non-fatal problems, reporting each as a warning.

Device required: no.

firmware inspect

Show a firmware binary's version, board type, MCU, and embedded ROM images and metadata.

```
onerom firmware inspect --firmware firmware.bin
```

For a fire-24-e image with the USB plugin and two 2364s:

```
Version:  0.8.0
Board:    fire-24-e
MCU:      RP235xA
Plugins:
  https://images.onerom.org/plugins/system/usb/v0.3.1/plugin.bin
Slots: 2
  Slot 0: 1 ROM(s), 8192 bytes
    Chip 0: 2364 kernal.bin
  Slot 1: 1 ROM(s), 8192 bytes
    Chip 0: 2364 basic.bin
```

From v0.7.0, release firmware is board-agnostic firmware. It runs on all boards and doesn't contain ROM images. Inspecting it prints `Board: any (base firmware)`.

For a One ROM Lab image:

- `Firmware: One ROM Lab`
- `Version:` its version
- `Board:` the board it was built for, or `(not set)`

Option	Description
`--firmware <FILE>` (aliases `--fw`, `--in`, `--input`)	Firmware binary to inspect.
`--board, -b <BOARD>`	Inspect the release firmware for this board type. Conflicts with `--firmware`.
`--version <VERSION>`	Firmware version to inspect. Defaults to latest. Conflicts with `--firmware`.

firmware releases

List available firmware releases with supported boards and MCUs.

```
onerom firmware releases
```

Option	Description
`--board, -b <BOARD>`	Show only releases for this board type.

© 2026 Piers Finlayson · onerom.org · MIT licence

Option	Description
`--all, -a`	Show all releases even if a device is attached and detected. Conflicts with `--board`.

firmware download

Download the base (ROM-less) firmware binary for a version/board/MCU.

```
onerom firmware download --version 0.6.5 --board fire-24-e --out firmware.bin
```

Option	Description
`--version <VERSION>`	Version to download. Defaults to latest.
`--board, -b <BOARD>`	Target board type. Inferred from device if omitted.
`--output, -o <FILE>` (alias `--out`)	Output file path. Defaults to `onerom_<board>_v<version>.bin`. Conflicts with `--path`.
`--path <DIR>`	Output directory, using the default filename. Conflicts with `--output`.

This firmware binary can then be used as a base for `firmware build` or flashed with `program` using the `--base-firmware` option. Do not flash it directly, as it contains no ROM configuration and will not serve any ROMs.

firmware chips

List the chip types a board can emulate and the flash each one uses, or all chip types grouped by pin count. Identical to the top-level `chips`.

```
onerom firmware chips --board fire-24-e
onerom firmware chips --board fire-24-e --chip-type 2364
onerom firmware chips --all
```

Option	Description
`--board, -b <BOARD>`	Show supported chips for this board. Conflicts with `--all`.
`--all, -a`	Show all chips grouped by pin count. Conflicts with `--board`.
`--chip-type, -c <CHIP>`	Show just this chip type's flash usage on the board. Conflicts with `--all`.

© 2026 Piers Finlayson · onerom.org · MIT licence

plugin

List available plugins from the release manifest, with versions and minimum firmware requirements. Without a connected device or `--fw-version`, minimum firmware requirements are shown for reference; with either, incompatible plugins are flagged.

```
onerom plugin
onerom plugin --all-versions
onerom plugin --type system
onerom plugin --fw-version 0.6.6
```

Option	Description
`--all-versions, -a`	Show all versions of each plugin, not just the latest.
`--type, -t <TYPE>`	Filter by plugin type: `system` or `user`.
`--fw-version <VERSION>`	Firmware version to check compatibility against.

Device required: no.

© 2026 Piers Finlayson · onerom.org · MIT licence

chips

List supported chip types — for a board, with the flash each one uses, or all grouped by pin count. Top-level alias for `firmware chips`.

```
onerom chips --board fire-24-e
onerom chips --board fire-24-e --chip-type 2364
onerom chips --all
```

Option	Description
`--board, -b <BOARD>`	Show supported chips for this board. Conflicts with `--all`.
`--all, -a`	Show all chips grouped by pin count. Conflicts with `--board`.
`--chip-type, -c <CHIP>`	Show just this chip type's flash usage on the board. Conflicts with `--all`.

Flash usage

For a board, each chip is listed with its **ROM size** (the chip's own capacity) and its **image size** — the flash One ROM uses to emulate it, which is often larger, and occasionally much larger. The figures, and the grouping, are the same ones published in Chip Compatibility; the document is generated from the same source the CLI reads, so the two agree.

Chips are grouped by how they fit the board's socket, and the **Fit** column names the fit exactly:

Fit	Meaning
`native`	Chip and board have the same pin count — it goes straight in.
`overhang`	Chip has *fewer* pins than the board, so One ROM's top pins hang out of the socket.
`larger socket (no fly-leads)`	Chip has *more* pins than the board, but no address line among the extra ones: One ROM sits bottom-justified in the socket, with nothing to wire.
`fly-lead to X1` / `fly-lead to X1 and X2`	Chip has more pins than the board, and the overhanging address line(s) must be wired to One ROM's `X1` (and `X2`) header pin.

Every fit other than `native` is a cross-size fit, and in all of them One ROM's power pins may not line up with the socket's — **power must be rerouted to One ROM's own VCC/5V pin**. `larger socket (no fly-leads)` means no *signal* wiring is needed. It doesn't mean the chip simply drops in. Use `board socket` with `--chip-type` and `--gpio` to see exactly where One ROM's VCC lands.

The sizes are for a chip served alone in its slot. A banked or multi-ROM set draws further lines into the slot's address window, so its image can be larger than the figure shown here; build the firmware and run `onerom firmware inspect` to see what a specific set costs.

Chips are listed only where a size can be derived, which means Fire (RP2350) boards. An Ice (STM32) board falls back to a plain list of names. A chip type of the board's own pin count that the board cannot serve — either because no firmware serves it yet (the SRAM types, at the time of writing) or because this particular board's layout cannot place it — is named in a trailing line rather than tabulated.

Board is taken from `--board`, or inferred from a connected One ROM. `--chip-type` accepts any chip type the board can emulate, under any accepted spelling.

© 2026 Piers Finlayson · onerom.org · MIT licence

Example output (illustrative — your build may differ):

```
$ onerom chips --board fire-24-f
Supported chip types for fire-24-f (One ROM Fire 24 (rev F)):

  24-pin chips (native)
    Chip       ROM size  Image size  Fit
    2704           512B        512B  native
    2364            8KB         8KB  native
    ...

  28-pin chips (with fly-leads)
    Chip       ROM size  Image size  Fit
    2764            8KB        32KB  fly-lead to X1
    ...

  Image size is the flash One ROM uses to emulate the chip, which may exceed the chip's own ROM
size.  See docs/COMPATIBILITY.md.

  Recognised but not servable on this board: 2016, 6116

Supported plugin types:
  SystemPlugin, UserPlugin
```

A single chip type, on the board that makes the point — an 8KB ROM costing 256KB of flash, because One ROM overhangs a 28-pin socket to emulate a 24-pin part:

```
$ onerom chips --board fire-28-c --chip-type 2364
2364 on fire-28-c (One ROM Fire 28 (rev C)):
  ROM size    8KB
  Image size  256KB
  Fit         overhang
```

With `--all`, chip types are listed by pin count without sizes, which are board-dependent:

```
Supported 24-pin chips:
  2016, 2316, 2332, 2364, 2704, 2708, 2716, 2732, 27C32, 28C16, 4732, 4764, ...
Supported 28-pin chips:
  231024, 23128, 23256, 23512, 23C1000, 23QL384, 23QL512, 27128, 27256, ...
Supported 32-pin chips:
  23C1001, 23C1010, 27C010, 27C020, 27C040, 29F010, 39SF010, SST39SF040, ...
Supported 40-pin chips:
  23C4100, 27C200, 27C200Pin31NC, 27C400, 27C400Pin31A17, 27C4100, AT27C400, ...
```

Device required: no (a device is used only to infer the board when `--board` is omitted).

© 2026 Piers Finlayson · onerom.org · MIT licence

board

List supported One ROM board types, or draw a board's physical pin layouts as ASCII.

```
onerom board <COMMAND>
```

Subcommand	Purpose	Device required
`list`	List the supported board types	No
`header`	Draw a board's pin (jumper) header	No
`socket`	Draw a board's ROM socket pinout	No

board list

Lists the board types, in two groups. This replaces the bare `onerom boards` of earlier releases, which no longer exists.

The first group is the boards the CLI can act on — the Fire (RP2350) boards. The second is the Ice (STM32) boards, which the CLI recognises but cannot scan, program or build firmware for; naming one on a command that needs a device or an image is an error. Commands that only report hardware still take them.

```
onerom board list
```

Example output (illustrative — your build may differ):

```
Supported One ROM board types:
  fire-24-a, fire-24-c, fire-24-d, fire-24-e, fire-24-eadb01, fire-24-f, fire-24-g, fire-24-usb-
b, fire-28-a, fire-28-b, fire-28-c, fire-28-d, fire-32-a, fire-32-b, fire-32-c, fire-40-a,
fire-40-b, fire-40-c

Recognised, but not supported by the CLI:
  ice-24-d, ice-24-e, ice-24-f, ice-24-g, ice-24-i, ice-24-j, ice-24-usb-h, ice-28-a
  These boards use an STM32, rather than the RP2350 the CLI works with.
```

`onerom scan --list-boards` prints the same listing.

Device required: no.

board header

Draw a board's pin (jumper / programming) header — the 2xN header along the board's top edge — as ASCII, pin by pin. Each image-select and X pin is annotated with the MCU GPIO wired to it, and on RP2350 (Fire) boards with whether that GPIO is 5V-tolerant (`5V`) or 3.3V-only (`!!3V3!!` — an ADC pin that must not be driven above 3.3V). See Voltage Levels for the ADC caveat.

```
onerom board header [--board <board>]
```

Option	Description
`--board`, `-b`	Board type to draw (e.g. `fire-24-f`). Inferred from a connected One ROM if omitted.

© 2026 Piers Finlayson · onerom.org · MIT licence

A board with no pin-header descriptor prints a short notice instead of a diagram.

```
onerom board header --board fire-24-f
```

Without `--board` the board type is read from the connected One ROM.

```
$ onerom board header --unrecognised
Failed to execute command.
Could not determine board type from the connected device Unknown          - Firmware: n/a
State: Stopped Serial: DE3F9C232F655B6B.
  It may be an unprogrammed One ROM or have corrupt firmware.
  Supply the board type with --board
```

The header carries the `BOOTSEL` pin used to boot a One ROM into its own bootloader — see Recovering a bricked One ROM.

Device required: no (a device is used only to infer `--board` when it is omitted).

board socket

Draw a board's ROM socket pinout as a DIP diagram.

```
onerom board socket [--board <board>] [--chip-type <chip>] [--gpio]
```

Option	Description
`--board`, `-b`	Board type to draw (e.g. `fire-24-f`). Inferred from a connected One ROM if omitted.
`--chip-type`, `-c`	Label pins with this chip type's ROM functions instead of GPIOs.
`--gpio`	Overlay the GPIO(s) behind each pin onto the `--chip-type` view. Requires `--chip-type`.

Without `--chip-type`, each socket pin is labelled with the GPIO(s) behind it (the GPIO map). With `--chip-type <chip>` (e.g. `2364`), the pins are labelled with that ROM's functions (address / data / chip-select / `BYTE` / power / ...) instead; add `--gpio` to overlay both. `--gpio` requires `--chip-type`. A pin that carries two functions on a multiplexed part (e.g. the 27C400's pin 29, `A0/D15`) shows both.

The two views that label pins with GPIOs — no `--chip-type`, or `--gpio` — need the board's GPIO map. A board without one reports that and draws nothing; the `--chip-type` function view still works, as it is drawn from the chip's pinout and the board's ROM signal assignments.

When the chip's pin count differs from the board's, the socket is drawn at the larger of the two and the smaller device is bottom-justified (see Chip Compatibility):

- emulating a **smaller** ROM on a larger One ROM, One ROM's extra pins hang out of the socket and are marked `overhang` (reroute power to One ROM's VCC/5V pin);
- emulating a **larger** ROM on a smaller One ROM, the socket pins One ROM does not reach are marked `(empty)`, and any address line there shows the One ROM `X1` / `X2` header pin it must be fly-leaded to (e.g. `A12 → X1`).

In both cases One ROM's own power pins may not line up with the ROM's. With `--gpio`, the pin One ROM's VCC (or GND) lands on is annotated `(VCC)` / `(GND)` — e.g. `NC (VCC)` shows One ROM's VCC sitting on the ROM's NC pin — so you know where power must be applied.

© 2026 Piers Finlayson · onerom.org · MIT licence

With `--chip-type`, the diagram is followed by that chip's image size on this board — the flash One ROM uses to emulate it, as reported by `onerom chips`.

`--chip-type` must be a chip type the board can emulate (native, overhang or fly-lead; see `onerom chips` and Chip Compatibility).

```
onerom board socket --board fire-24-f
onerom board socket --board fire-24-f --chip-type 2364
onerom board socket --board fire-24-f --chip-type 2364 --gpio
```

Device required: no (a device is used only to infer `--board` when it is omitted).

© 2026 Piers Finlayson · onerom.org · MIT licence

self

Check for, and download, new releases of the CLI itself. This is the CLI's own release channel — the binaries published at https://onerom.org/cli — and is separate from the One ROM firmware releases `firmware releases` lists.

Nothing here runs unless you ask for it: the CLI performs no update check of its own accord, and neither command installs anything.

```
onerom self check
onerom self download
```

Device required: no.

self check

Compare this build against the newest release published for this platform.

```
onerom self check
```

This command takes no options.

It prints the running version, then one of three things: that this is the latest release; that a newer one is available, with how to get it; or — for a build made from source — that this build is newer than anything published.

Finding an update is not an error: the exit code is 0 in all three cases. A non-zero exit means the check itself failed, such as an unreachable images server, so a script can tell "no update" from "could not tell".

self download

Download a published CLI release.

```
onerom self download
onerom self download --version 0.3.0 --path ~/Downloads
onerom self download --target aarch64-unknown-linux-gnu
onerom self download --target all --path ./dist
```

Option	Description
`--version <VERSION>`	Version to download (e.g. `0.3.0`). Defaults to the latest release.
`--target <TARGET>`	Platform to download for, as a target triple. Defaults to this machine's. `all` downloads every platform's artifact for the version, and requires `--path`.
`--output, -o <FILE>` (alias `--out`)	Output file path. Defaults to the published filename. Conflicts with `--path`.
`--path <DIR>`	Output directory, using the published filename. Conflicts with `--output`.
`--force, -f`	Overwrite an existing file.

The downloaded file is checked against the SHA-256 published alongside it; a mismatch is reported and the download discarded. That digest comes from the same server as the file, so it catches a corrupted

or truncated download rather than a compromised server — it is not a signature. The Windows and macOS builds are digitally signed, and that signature is what your OS checks when you run them.

The published filename already carries the version and architecture, so it is the default name in both the current directory and a `--path` directory. Every output path is checked before anything is downloaded, so an existing file or a missing directory fails immediately rather than part-way through a `--target all` run.

Platform names are the Rust target triples the manifest publishes: `x86_64-unknown-linux-gnu`, `aarch64-unknown-linux-gnu`, `x86_64-pc-windows-msvc`, `aarch64-pc-windows-msvc`, and `universal-apple-darwin`. The last is not a real triple: the macOS build is a universal binary covering both Apple architectures. Naming an unknown one lists what is published.

`--target` is there to fetch a build for a machine other than this one — the ARM `.deb` for a Raspberry Pi, say, from a desktop. When the artifact is not for this platform, no install step is printed, since the local one would not apply.

Nothing is installed and the running `onerom` is not replaced. Install what was downloaded the same way you would install it from the website — see Installation.

© 2026 Piers Finlayson · onerom.org · MIT licence

Top-level aliases

Convenience aliases for frequently used nested commands. They take the same options as their targets.

peek (top-level alias)

Alias for `inspect peek live`.

```
onerom peek --address 0x100 --length 64
```

poke (top-level alias)

Alias for `control poke live`.

```
onerom poke --address 0x100 --input patch.bin
```

reboot (top-level alias)

Alias for `control reboot`.

```
onerom reboot
```

© 2026 Piers Finlayson · onerom.org · MIT licence

ROM slot specification

Used by `--slot` in `program` and `firmware build`. Repeat `--slot` once per slot. Comma-separated `key=value` pairs:

```
file=<path_or_url>,type=<romtype>[,label=<text>]
    [,cs1=<logic>][,cs2=<logic>][,cs3=<logic>][,cs4=<logic>]
    [,size-handling=<handling>][,format=<binary|ihex|srec>][,load-address=<addr>]
    [,transform=<list>]
    [,cpu-freq=<freq>][,cpu-vreg=<voltage>][,led=<bool>][,force-16-bit=<bool>]
    [,standby=<bool>][,addr-mon-min-cs-act-ns=<ns>]
```

Key	Values / notes
`file`	Local path or URL to the ROM image.
`label` (alias `name`)	A name for this image, recorded in the device metadata in place of the filename, and shown by `scan --slots`, `inspect slots` and `firmware inspect`. Worth setting when the file is a long path or URL, or when the recorded name would otherwise be truncated — see Image transforms. The same field is `label` in a config file.
`type`	Chip type, e.g. `2364`, `2332`, `2716`, `27C400`. Any type the target firmware can serve on the board is accepted — that is exactly what `chips --board` lists, including the overhang and fly-lead combinations (a `2764` on a Fire 24, say); see COMPATIBILITY.md. Building for firmware older than v0.7.0 accepts a narrower set, and a rejection lists what that firmware serves. Any accepted alias may be used; the exact spelling you enter is preserved in the device metadata (shown by `scan`/`inspect`), while the resolved type drives behaviour.
`cs1`, `cs2`, `cs3`, `cs4`	CS polarity: `active-low` (or `0`), `active-high` (or `1`), or `ignore`. The snake_case config spellings (`active_low`, `active_high`) are also accepted. Which lines are required depends on the chip type (e.g. `2332` requires `cs1` and `cs2`). A chip type without that line, or with its polarity fixed in silicon, rejects it — CHIP-TYPES.md lists each type's control lines. `ignore` says One ROM does not monitor the line at all — it is not a polarity, and is only permitted where the chip type or set allows it (see `allow_cs_ignore`).
`size-handling` (aliases `size`, `size_handling`)	`none`, `duplicate` (or `dup`), `truncate` (or `trunc`), `pad`. For an Intel HEX or S-record image, padding fills with `0xFF` and `duplicate` is not permitted.
`format`	`binary` (default), `ihex` (Intel HEX) or `srec` (Motorola S-record). An `ihex` or `srec` file is decoded to a binary image before use; unwritten bytes read as `0xFF`.
`load-address` (alias `load_address`)	Only valid with `format=ihex` or `format=srec`. The absolute address that maps to byte 0 of the ROM, as a decimal or `0x`/`$`-prefixed hex value (e.g. `$E000`). Defaults to `0`.
`transform`	Byte-level rearrangements of the image, applied in the order given and joined with `+`. See Image transforms.
`cpu-freq`	e.g. `150`, `150mhz`, `150MHz`. Values above 150 MHz require confirmation (suppressed by `--yes`) and set overclock automatically.
`cpu-vreg`	e.g. `1.1`, `1.10`, `1.10v`, `1.10V`. Values above 1.10 V require confirmation (suppressed by `--yes`). Must be a supported level.
`led`	Boolean: `on`/`off`, `true`/`false`, `1`/`0`.

© 2026 Piers Finlayson · onerom.org · MIT licence

Key	Values / notes
`force-16-bit` (alias `force_16bit`)	Boolean (as above). Valid only on 40-pin boards.
`standby`	Boolean (as above). Controls whether One ROM boots into standby mode when this slot is selected. In standby mode One ROM doesn't serve the ROM. With `standby=on`, `file` is optional. Requires firmware v0.8.0 or later. See `control standby`.
`addr-mon-min-cs-act-ns` (alias `addr_mon_min_cs_act_ns`)	A number of nanoseconds without a unit. Overrides `--addr-mon-min-cs-act-ns` when this slot is selected. Requires firmware v0.8.1 or later.

Examples:

```
--slot file=kernal.bin,type=2364,cs1=active-low
--slot file=chargen.bin,type=2332,cs1=active-low,cs2=active-high
--slot file=https://example.com/basic.bin,type=2716
--slot file=https://example.com/c64/roms/901227-03.bin,type=2364,cs1=active-low,label=kernal
--slot file=small.bin,type=2364,cs1=active-low,size-handling=duplicate
--slot file=kernal.hex,type=2364,cs1=active-low,format=ihex
--slot file=kernal.hex,type=2364,cs1=active-low,format=ihex,load-address=$E000
--slot file=kernal.s19,type=2364,cs1=active-low,format=srec,load-address=$E000
--slot file=kernal.bin,type=2364,cs1=active-low,cpu-freq=200MHz,cpu-vreg=1.2V
--slot file=char.bin,type=2332,cs1=active-low,cs2=active-high,led=off
--slot file=amiga.bin,type=27C400,force-16-bit=true
--slot type=2364,cs1=active-low,standby=on
--slot file=kernal.bin,type=2364,cs1=active-low,addr-mon-min-cs-act-ns=140
--slot file=undersized.bin,type=2732,size=pad
--slot file=oversized.bin,type=2732,size=trunc
--slot file=halfsized.bin,type=2732,size=dup
--slot file=amiga.bin,type=27C400,transform=swap_bytes
--slot file=rom32.bin,type=27C010,transform=deinterleave:1/2/2+swap_bytes
```

© 2026 Piers Finlayson · onerom.org · MIT licence

Image transforms

Some ROM images are not laid out the way the target chip needs them: a 16-bit part whose image was produced with the opposite byte order, or a wide image that interleaves several narrower devices. Transforms rearrange the bytes before the image is written into the firmware.

They are available two ways, and both run exactly the same operation:

- as the `transform=` key of a `--slot`, or a `"transform"` array in a config file, applied during the build;
- as the standalone `image swap-bytes` and `image deinterleave` subcommands, which rewrite a file.

Transform	Effect
`swap_bytes`	Reverses the byte order within each 16-bit word. The image must have an even length.
`deinterleave:<offset>/<stride>`	The image contains `stride` interleaved lanes one byte wide; keep lane `offset`.
`deinterleave:<offset>/<stride>/<bytes>`	As above, with lanes `bytes` wide — use `2` to keep 16-bit words together.

The parameters are positional, in the order `<offset>/<stride>/<bytes>`, and `<bytes>` may be omitted (it defaults to `1`).

Each name has aliases, accepted identically by the CLI and by a config file:

Canonical	Also accepted
`swap_bytes`	`swap-bytes`, `swapbytes`
`deinterleave`	`de_interleave`, `de-interleave`, `deint`
`transform=` (slot key)	`trans=`

`deinterleave` requires the image length to be a multiple of `bytes × stride` — one full set of lanes. The result is `1/stride` of the input length.

Transforms are applied **in the order listed**, and the order matters:

```
transform=deinterleave:1/2/2+swap_bytes
```

takes the upper 16-bit half of each 32-bit word and *then* swaps its byte pairs. Note that `offset` selects which lane, not a named "high" or "low" half — which half you get depends on the byte order of your source image, and that is what `swap_bytes` is for.

Within the build pipeline, transforms run after any `location` window and after an Intel HEX or S-record image has been decoded, but before `size-handling` reconciles the image against the chip size. A `swap_bytes` on an odd-length image is an error unless `size-handling` is `pad` (which appends one blank byte) or `truncate` (which drops the trailing byte). Where the size handling is used this way it counts as having been needed, so it is not then reported as redundant even if the transformed image lands on exactly the chip size.

Common recipes:

© 2026 Piers Finlayson · onerom.org · MIT licence

Goal	Spec
16-bit image with the wrong byte order	`transform=swap_bytes`
Even / odd bytes of a 16-bit interleaved image	`transform=deinterleave:0/2` / `transform=deinterleave:1/2`
Byte *n* of a 32-bit interleaved image	`transform=deinterleave:<n>/4`
One 16-bit half of a 32-bit interleaved image	`transform=deinterleave:0/2/2` or `deinterleave:1/2/2`

The transforms applied to an image are recorded in the firmware metadata alongside its filename — as `kick.bin|transform=swap_bytes` — so a built image carries a record of how its ROM data was derived. Note that the metadata filename field is capped at 128 bytes, so the suffix can be truncated away for a very long path; use `label=` to keep it short.

16-bit ROM image byte ordering

A 16-bit ROM supplies two bytes at a time, and an image of one may hold each pair in either order. One ROM reads the low byte of each pair first. An image holding the high byte first needs `swap_bytes`, and without it every pair is served reversed and the machine does not work.

The CLI checks for this. It compares the first bytes of the image against a list of known ROM headers, including ones seen in Amiga Kickstart, DiagROM and Atari ST TOS images. An image matching no entry is left alone.

A 16-bit image holding the high byte of each pair first, with no transform:

```
$ onerom firmware build --board fire-40-a --slot file=kick.bin,type=27C400 --out fw.bin
Warning: kick.bin starts with an Amiga ROM header, high byte of each pair first.
  One ROM needs the low byte of each pair first.  Add transform=swap_bytes to
  this slot.
```

`swap_bytes` applied to an image that did not need it:

```
$ onerom firmware build --board fire-40-a --slot file=kick-
swapped.bin,type=27C400,transform=swap_bytes --out fw.bin
Warning: kick-swapped.bin starts with an Amiga ROM header and was already the way One
  ROM needs it.  transform=swap_bytes has swapped it the wrong way round.
  Remove transform=swap_bytes from this slot.
```

Neither is refused. The build and the programming go ahead.

The check is skipped for an 8-bit chip, whose image holds no 16-bit words, and for a slot carrying any transform besides `swap_bytes`, since the check then reads bytes that are not the ones served.

`--verbose` also reports a recognised image that needs no change, and one the check could not identify:

```
$ onerom --verbose image swap-bytes --input kick.bin --output out.bin
kick.bin starts with an Amiga ROM header, high byte of each pair first.
  Swapping the bytes makes it correct for One ROM.
```

© 2026 Piers Finlayson · onerom.org · MIT licence

```
$ onerom --verbose firmware build --board fire-40-a --slot file=blank.bin,type=27C400 --out
fw.bin
Unable to tell which way around the byte pairs are in blank.bin.  If the slot
  does not work, try transform=swap_bytes.
```

© 2026 Piers Finlayson · onerom.org · MIT licence

Plugin specification

At most one system plugin and one user plugin are supported. A user plugin requires a system plugin. The system plugin is placed in slot 0, the user plugin in slot 1.

Various commands expose the `--plugin` option:

Form	Meaning
`--plugin usb`	Latest compatible version of a published plugin.
`--plugin system/usb`	With explicit type (`system` or `user`).
`--plugin usb,version=0.1.0`	Pinned version.
`--plugin file=path/to/plugin.bin`	Local file.
`--plugin file=https://example.com/plugin.bin`	Remote file.

See also Plugin compatibility.

© 2026 Piers Finlayson · onerom.org · MIT licence

Pin values

A pin is written in one of these forms. Case is ignored.

Form	Pin
`sel_<letter>`	An image select pin, for example `sel_a`. `sel-a` and `sela` also work.
`x1`, `x2`	An X pin.
`gpio<N>`	An MCU GPIO, for example `gpio23`.

Used by:

- `--pin` in `control pin`, `control reset` and `inspect gpio`
- `--reset-host` in `program`
- `--reserve-pin` in `program` and `firmware build`

Only image select pins and X pins can be reserved.

A header pin's GPIO depends on the board. The board is read from the connected One ROM. Use `--board` if its board isn't recognised.

Use `onerom inspect header` to see each header pin's GPIO.

© 2026 Piers Finlayson · onerom.org · MIT licence

Problems

Recovering a bricked One ROM

You can recover a One ROM that is not responding using any of the One ROM programming tools by following the instructions below. The One ROM CLI is recommended as it gives greatest control over One ROM. The CLI commands are shown below.

Situation

No tool can find the device. `onerom scan` reports nothing, the browser programmer sees nothing to connect to, and any command needing a device refuses. The Web programmer cannot detect it and the CLI reports:

```
$ onerom scan
Scanning ...
No matching One ROM devices found.

$ onerom inspect info
Failed to execute command.
No One ROM was found or specified.
  Specify a One ROM using --serial.
  Use 'onerom scan' to list connected One ROMs.
```

A One ROM in this state is called bricked. Nothing is damaged. Its firmware is not running, so nothing answers on the USB bus — programming was interrupted, or the firmware on it is not right for the board. One ROM has a hardware bootloader which cannot be bricked, so the recovery is to boot the device into that bootloader and program it again.

If the One ROM programming tool you are using does find the device, it is not bricked. Program it as normal.

Booting into the bootloader

This works on any Fire (RP2350) board whatever state its flash is in.

1. Unplug the One ROM.
2. Connect the **BOOTSEL** pad to ground. It is normally the middle pad/pin on the header pins' top row, and the USB shield is a good source of ground. The CLI command `onerom board header --board <BOARD>` shows the header pins.

   > Fire 24 rev A and Fire 24 USB rev B are the exceptions, and both are rare. Rev A brings BOOTSEL out as a pin towards the bottom of the board, and USB rev B as a small pad on the underside.

3. Plug the One ROM into USB with that connection still made. The status LED lights dimly, which is how you know the bootloader is running.
4. Remove the BOOTSEL to ground connection — it is needed only as power comes up.

© 2026 Piers Finlayson · onerom.org · MIT licence

Checking the host can see it

Connect to One ROM using the programming tool as normal.

Using the CLI `--unrecognised` (`-u`) matches any attached RP2350 board, a including a Raspberry Pi Pico 2, so make sure only the One ROM is attached:

```
$ onerom scan --unrecognised
Scanning ...
found 1 connected device:
  Unknown           - Firmware: n/a   State: Stopped Serial: DE3F9C232F655B6B
```

`scan` reads a device's board and firmware version from its firmware. Where it can't read the firmware it reads the board from the device's commissioning data if present.

Programming it again

You have to supply the One ROM board information unless the board has been commissioned. An uncommissioned One ROM's board type is only identifiable from the firmware already programmed to it, and that either missing or wrong. The board name is the pin count and the revision letter silkscreened on the board — `fire-24-f` is a 24-pin board, revision F. The marking is small. `onerom board list` prints every name.

To re-program with the CLI, add `--unrecognised` and `--board`:

```
onerom program --unrecognised --board fire-24-f --config c64.json
```

With the browser programmer, pick the board yourself in the same way. It will ask you to confirm the board type before it writes.

If an uncommissioned board was mis-flashed rather than left blank, the browser programmer reads the board from the wrong firmware still in flash. A warning appears and you can continue. Check the silkscreen once more before you do, because the same warning appears when the board is right and the name you picked is wrong.

Then confirm the device came back up. With the CLI:

```
onerom scan
```

Getting the board wrong writes the wrong firmware and leaves you with a bricked device. In this case, follow the instructions again.

Ice boards

Ice (STM32) boards use `BOOT0` rather than BOOTSEL, and it is pulled **high**, to 3.3V, rather than to ground. It is the jumper labelled `B0` or `B`. The 0.7.x CLI does not program Ice boards at all — use the Web Programmer or One ROM Studio.

© 2026 Piers Finlayson · onerom.org · MIT licence

Appendix: Breaking Change History

Changes that can alter or break a command line that worked on an earlier release. Newest release first.

v0.6.0

- `program` and `firmware build` automatically add the USB plugin when a system plugin isn't specified. Use `--no-plugin` to build without any plugins.
- An unused signature from an earlier CLI fails with `hardware commission --signature`. One ROMs already commissioned aren't affected. Raise a new signing request, or sign again with `hardware sign`.
- `hardware commission` and `hardware sign` fail with a manufacturer name longer than 32 characters.
- `--config-name`, `--config-description`, `--save-config` and the per-device overrides are rejected when used with `program --firmware`. When used with `program --base-firmware` these, `--plugin` and `--no-plugin` are rejected unless `--config` or `--slot` is used. Previously they were ignored.
- `program --base-firmware` without `--config` or `--slot` requires `--no-config`.
- `console --no-echo` requires `--raw`.

v0.5.0

- One ROM Lab is now recognised by the CLI, so with a One ROM and a Lab both connected every command that uses a device, except `scan`, now needs `--serial` to disambiguate the devices.

v0.4.0

- `--name` is an alias for `--instance-name` on `program` and `firmware build`. It was an alias for `--config-name`. A command line using `--name` alongside `--slot` still runs, and names the One ROM rather than the configuration. With `--no-config` it is now rejected.

v0.3.0

- `onerom boards` is now `onerom board`, and the bare listing it printed is `onerom board list`. There is no alias.
- `onerom control erase` takes `--stopped` / `--running` for its post-erase reboot mode, in place of `--reboot-stopped` / `--reboot-running`. There is no alias.
- `--allow-unsupported-chip-type` is removed from `program` and `firmware build`. Every chip type the target firmware can serve is accepted without it.
- No command takes a positional argument. `board header` and `board socket` take `--board`.
- Each short flag means one thing across the whole CLI: `-b` is `--board` (was `--byte` on `poke`), `-o` is `--output` (was `--offset` on `control erase`), `-i` is `--input` (was the global `--vid-pid`, which keeps `--id`), `-l` is `--length` (was `--slot`), and `-m` is `--msd` (was `--image` on `update slot`).
- `firmware build` no longer accepts `--swd_disabled`. `--disable-swd` and `--swd-disable` work on both it and `program`.

© 2026 Piers Finlayson · onerom.org · MIT licence